A

Major Project

On

A COMPARATIVE HYBRID DEEP LEARNING STUDY OF TWITTER SENTIMENT ON THE RUSSIA-UKRAINE WAR

(Submitted in partial fulfilment of the requirements for the award of Degree)

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY

By

G.Praneetha (227R1A1220)

Ch.Sridhar (227R1A1212)

V.Vamshi (227R1A1260)

Under the Guidance of

Mrs.M.SIVAJYOTHI

(Assistant Professor)

DEPARTMENT OF INFORMATION TECHNOLOGY

CMR TECHNICAL CAMPUS

UGC AUTONOMOUS

(Accredited by NAAC, NBA, Permanently Affiliated to JNTUH, Approved by AICTE, NewDelhi)

Recognized Under Section 2(f) & 12(B) of the UGCAct.1956,

Kandlakoya (V), Medchal Road, Hyderabad-501401.

May, 2026.

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the project entitled "**A COMPARATIVE HYBRID DEEP LEARNING STUDY OF TWITTER SENTIMENT ON THE RUSSIA UKRAINE WAR"** being submitted by **G.Praneetha (227R1A1220), Ch.Sridhar (227R1A1212) & V.Vamshi (227R1A1260)** in partial fulfilment of the requirements for the award of the degree of B.Tech in Information Technology to the Jawaharlal Nehru Technological University Hyderabad, during the year 2025-26.

The results embodied in this thesis have not been submitted to any other University or Institute for the award of any degree or diploma.

Mrs.M.Sivajyothi
Assistant Professor
INTERNAL GUIDE

HoD

Dr. A. Raji Reddy
DIRECTOR

Signature of External Examiner

Submitted for viva voice Examination held on _____

ACKNOWLEDGEMENT

We take this opportunity to express our gratitude to the people who have been instrumental in the successful completion of this project, we take this opportunity to express our profound gratitude and deep regard to our guide **Mrs.M.Sivajyothi,** Assistant Professor for her exemplary guidance, monitoring and constant encouragement throughout the project work. The blessing, help and guidance given by her shall carry us a long way in the journey of life on which we are about to embark.

We take this opportunity to extend our heartfelt appreciation to the Project Review Committee (PRC) Coordinators — **Dr. J. Narasimharao, Mr. Md. Sajid Pasha, Mr. K. Praveen Kumar, Mr. D. Krishna Kumar, Mr. Y. Satyam and Mr. K. Srinu** —for their unwavering support, insightful guidance, and valuable inputs, which played a crucial role in steering this project through its various stages.

Our sincere appreciation also goes to our **Head of the Department**, for encouragement and continuous support in ensuring the successful completion of our project.

We are deeply grateful to **Dr. A. Raji Reddy**, Director, for his cooperation throughout the course of this project. Additionally, we extend our profound gratitude to Sri. **Ch. Gopal Reddy**, Chairman, Smt. **C. Vasantha Latha**, Secretary and Sri. **C. Abhinav Reddy**, Vice- Chairman, for fostering an excellent infrastructure and a conducive learning environment that greatly contributed to our progress.

We also acknowledge and appreciate the guidance and assistance provided by the faculty and staff of **CMR Technical Campus**, whose contributions have been invaluable in bringing this project to fruition.

Lastly, we sincerely thank our families for their unwavering support and encouragement. We also extend our gratitude to the teaching and non-teaching staff of CMR Technical Campus for their guidance and assistance. Their contributions, along with the support of everyone who helped directly or indirectly, have been invaluable in the successful completion of this project.

G.Praneetha (227R1A1220)
Ch.Sridhar (227R1A1212)
V.Vamshi (227R1A1260)

VISION AND MISSION

INSTITUTE VISION:

To Impart quality education in serene atmosphere thus strive for excellence in Technology and Research.

INSTITUTE MISSION:

1. To create state of art facilities for effective Teaching- Learning Process.

2. Pursue and Disseminate Knowledge based research to meet the needs of Industry & Society.

3. Infuse Professional, Ethical and Societal values among Learning Community.

DEPARTMENT VISION:

To produce technically proficient and skilled graduates in Information Technology and to prepare them to meet the needs of industry and society in a holistic learning environment.

DEPARTMENT MISSION:

1. To prepare students with the essential knowledge and contemporary abilities for solving emerging technological problems.

2. To train the students on cutting – edge computing facilities with industry interaction to enhance in practical exposure to real world situations.

3. To enhance the leadership qualities and team spirit with social values to serve the society.

ABSTRACT

This project is titled as “A Comparative Hybrid Deep Learning Study of Twitter Sentiment on the Russia-Ukraine War”. Over 500 million tweets are posted daily, with nearly 30% relating to global socio-political issues such as the Russia-Ukraine War. Traditional manual sentiment analysis is time-consuming and highly prone to human error, making it unsuitable for real-time public opinion tracking. The pipeline begins with Natural Language Processing (NLP) preprocessing techniques including tokenization, stopword removal, and lemmatization, followed by Exploratory Data Analysis (EDA) to identify sentiment distribution and textual patterns. Distilled Robustly Optimized BERT Pretraining ApproaDistil RoBERTa) is employed for high-dimensional feature extraction due to its efficiency in handling contextual embeddings. To address class imbalance, the model integrates KMeans-SMOTE (Synthetic Minority Oversampling Technique), ensuring balanced representation across sentiment classes: Negative, Neutral, and Positive. For comparative analysis, existing classifiers such as Logistic Regression (LR), Support Vector Machine (SVM), and Random Forest Classifier (RFC) were implemented. The proposed framework enhances classification through Deep Neural Network (DNN)-based feature selection combined with a Greedy Tree mechanism, significantly optimizing sentiment categorization. Results demonstrate that the hybrid model outperforms existing baselines, providing a scalable and robust solution for real-time sentiment monitoring of conflict-related discourse.

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
Figure 3.1	Project Architecture of A Comparative Hybrid Deep Learning Study of Twitter Sentiment on the Russia-Ukraine War.	12
Figure 3.3	Use case Diagram of A Comparative Hybrid Deep Learning Study of Twitter Sentiment on the Russia-Ukraine War.	15
Figure 3.4	Class Diagram of A Comparative Hybrid Deep Learning Study of Twitter Sentiment on the Russia-Ukraine War.	16
Figure 3.5	Sequence Diagram of A Comparative Hybrid Deep Learning Study of Twitter Sentiment on the Russia-Ukraine War.	17
Figure 3.6	Activity Diagram of A Comparative Hybrid Deep Learning Study of Twitter Sentiment on the Russia-Ukraine War.	18
Figure 3.7	Dataflow Diagram of A Comparative Hybrid Deep Learning Study of Twitter Sentiment on the Russia-Ukraine War.	20
Figure 4.1	Word Cloud of the Top 100 Most Frequent Terms in the Russia-Ukraine War Twitter Dataset	25

Figure 5.1	Home page of sentiment analysis of Russia Ukraine war.	41
Figure 5.2	Registration page of sentiment analysis of Russia Ukraine war.	42
Figure 5.3	Login page of sentiment analysis of Russia Ukraine war.	43
Figure 5.4	Statistical Representation of Sentiment Classes in the Russia- Ukraine War Twitter Dataset.	43
Figure 5.5	Confusion Matrix obtained using Distil RoBERT- (a)LR Model (b)RFC Model.	45
Figure 5.6	Confusion Matrix obtained using Distil RoBERT- (c)SVC Model (d)DNN Greedy Tree Model.	45
Figure 5.7	ROC Curve obtained using Distil RoBERT- LR Model.	46
Figure 5.8	ROC Curve obtained using Distil RoBERT- RFC Model.	47
Figure 5.9	ROC Curve obtained using Distil RoBERT- DNN Greedy Tree Model.	48
Figure 5.10	Prediction page of the sentiment analysis of Russia-Ukraine War.	49

LIST OF TABLES

TABLE NO	TABLE NAME	PAGE NO
Table 6.2.1	Performance Evaluation	51

TABLE OF CONTENTS

ABSTRACT **i**
LIST OF FIGURES **ii**
LIST OF TABLES **iv**
1. INTRODUCTION **1**
1.1 PROJECT PURPOSE 1
1.2 PROJECT FEATURES 1
2. LITERATURE SURVEY **3**
2.1 REVIEW OF RELATED WORK 6
2.2 DEFINITION OF PROBLEM STATEMENT 8
2.3 EXISTING SYSTEM 8
2.4 PROPOSED SYSTEM 9
2.5 OBJECTIVES 10
2.6 HARDWARE & SOFTWARE REQUIREMENTS 11
2.6.1 HARDWARE REQUIREMENTS 11
2.6.2 SOFTWARE REQUIREMENTS 11
3. SYSTEM ARCHITECTURE & DESIGN **12**
3.1 PROJECT ARCHITECTURE 12
3.2 DESCRIPTION 13
3.3 USE CASE DIAGRAM 15
3.4 CLASS DIAGRAM 16
3.5 SEQUENCEDIAGRAM 17
3.6 ACTIVITY DIAGRAM 18
3.7 DATA FLOW DIAGRAM 20
4 IMPLEMENTATION **21**
4.1 ALGORITHMS USED 21
4.2 SAMPLE CODE 26
5 RESULTS & DISCUSSION **41**
6 VALIDATION **50**
6.1 INTRODUCTION 50
6.2 TEST CASES 51
6.2.1 PERFORMANCE EVALUATION 51
7 CONCLUSION & FUTURE ASPECTS **52**
7.1 PROJECT CONCLUSION 52
7.2 FUTURE ASPECTS 53
8 BIBLIOGRAPHY **54**
8.1 REFERENCES 54
8.2 GITHUB LINK 55

1. INTRODUCTION

1. INTRODUCTION

The Russia–Ukraine war has had a major global impact on political, economic, and social conditions, leading to widespread discussions on social media platforms such as Twitter. Millions of users share their opinions online, making sentiment analysis important for understanding public reactions. However, the large volume of tweets makes manual analysis difficult, and traditional methods cannot efficiently handle unstructured text data. Therefore, Natural Language Processing (NLP) and deep learning techniques are required for accurate and automated sentiment analysis.

This study proposes a comparative hybrid deep learning approach to analyze Twitter sentiment on the Russia–Ukraine war. The method combines NLP preprocessing, transformer-based feature extraction, and multiple machine learning classifiers to improve accuracy and handle noisy and imbalanced data. By comparing different hybrid models, the study provides reliable sentiment classification and helps in understanding global public opinion in real time.

1.1 PROJECT PURPOSE

The purpose of this project is to develop an automated system to analyze public sentiment about the Russia–Ukraine war using Twitter data. Since social media generates a large amount of unstructured text, traditional sentiment analysis methods are not efficient. Therefore, this project uses a hybrid deep learning approach with Natural Language Processing (NLP), preprocessing techniques, and pretrained models to classify tweets into positive, negative, and neutral sentiments accurately.

Another objective of this project is to compare different machine learning and deep learning models within a hybrid framework to improve accuracy and handle noisy and imbalanced data. The system helps in understanding global public opinion in real time and provides useful insights for researchers, analysts, and decision-makers during the Russia– Ukraine conflict

1.2 PROJECT FEATURES

This project incorporates several key features to improve the accuracy and efficiency of Twitter sentiment classification related to the Russia-Ukraine war. The system uses advanced Natural Language Processing (NLP) techniques to automatically analyze large volumes of Twitter data. Preprocessing steps such as tokenization, stopword removal, normalization, and

lemmatization are applied to clean the text. The processed data is then converted into contextual embeddings using the RoBERTa pretrained transformer model, which helps the system understand the meaning of words based on context and improves sentiment detection accuracy.

To handle the problem of imbalanced sentiment data, the system uses KMeans-SMOTE to generate synthetic samples for minority classes. This helps the model learn all sentiment categories equally and reduces bias during training. The system is also designed to manage noisy and unstructured social media text, including slang, abbreviations, and emotional expressions, which are commonly found in tweets related to the Russia-Ukraine war.

The system is designed to process large-scale Twitter datasets efficiently and can be used for real-time sentiment monitoring. It provides graphical outputs such as accuracy charts, confusion matrices, and sentiment distribution graphs to compare different models. These visualizations help researchers and analysts understand public opinion trends and evaluate the performance of the hybrid deep learning models more effectively.

2. LITERATURE SURVEY

2.LITERATURE SURVEY

Sentiment analysis has become an important research area in Natural Language Processing (NLP) and machine learning, especially for analyzing opinions expressed on social media platforms. With the rapid growth of Twitter and other online networks, researchers have focused on developing automated techniques to understand public opinion from large amounts of textual data. Traditional sentiment analysis methods mainly relied on manual labeling, keyword-based approaches, or simple machine learning algorithms. These approaches were useful for small datasets but were not suitable for analyzing large-scale social media data because tweets often contain slang, abbreviations, sarcasm, and context-dependent meanings. Manual analysis is slow and prone to human error, while rule-based systems cannot understand complex language patterns. Therefore, advanced machine learning and deep learning techniques are required to analyze sentiment accurately from social media data.

During major global events such as wars, elections, and pandemics, social media becomes an important source for understanding public opinion. The Russia–Ukraine war generated millions of tweets where people expressed their emotions, support, criticism, and concerns about the conflict. Researchers have used these tweets to study global sentiment and emotional reactions. Vyas et al. proposed a framework for classifying emotions related to the Russia–Ukraine war using transformer-based models and machine learning techniques. Their model used an Emoroberta-based approach to extract emotional features from tweets and classify them into multiple emotion categories. The study showed that most Twitter users expressed neutral opinions, while many tweets mentioned political leaders and countries involved in the conflict. This research demonstrated the importance of automated sentiment analysis systems for understanding public reactions during geopolitical events.

Another important development in sentiment analysis is the use of pretrained language models such as BERT and RoBERTa. Liu et al. proposed a sentiment classification model that combines RoBERTa with Convolutional Neural Networks (CNN). In this approach, RoBERTa is used to generate contextual embeddings that capture the semantic meaning of words, while CNN extracts important features for classification. The final sentiment is predicted using a softmax classifier. The study showed that combining pretrained models with deep learning techniques improves precision, recall, and F1-score compared to traditional methods. This approach is useful for analyzing social media text because it can understand complex language structures and contextual relationships between words.

Researchers have also studied different word-embedding techniques such as TF-IDF, Word2Vec, and FastText for text classification tasks. Al-Tarawneh et al. compared these embeddings with machine learning and deep learning models including Support Vector Machine (SVM), Multilayer Perceptron (MLP), and Convolutional Neural Networks (CNN). Their results showed that TF-IDF combined with SVM or CNN achieved better performance in many cases, while semantic embeddings like Word2Vec and FastText were useful for capturing meaning but sometimes increased model complexity. This study highlights that selecting the correct feature representation is very important for improving classification accuracy. It also suggests that hybrid models combining different techniques can produce better results than single-model systems.

Several studies have shown that deep learning models such as LSTM, CNN, and transformer-based networks are effective for sentiment analysis because they can learn complex patterns from text. Kim et al. used FinBERT and LSTM models to analyze sentiment from news articles and used the sentiment scores to predict stock prices. Their results showed that including sentiment information improves prediction accuracy. This study demonstrates that sentiment analysis is not only useful for social media research but also for financial forecasting and decision-making. Deep learning models are able to capture long-term dependencies in text and provide better performance than traditional machine learning algorithms.

Hybrid approaches that combine multiple models have also been widely studied in recent years. Frohmann et al. proposed a hybrid system that combines sentiment analysis with time-series forecasting to predict cryptocurrency prices. Their model used BERT-based sentiment analysis together with regression models to improve prediction accuracy. The results showed that hybrid models perform better than single models because they can capture both semantic information and numerical patterns. This idea is important for sentiment analysis of complex events such as wars, where emotions and opinions change rapidly over time.

Another important challenge in sentiment analysis is handling imbalanced datasets. In social media data, some sentiment classes may have more samples than others, which can reduce model accuracy. To solve this problem, techniques such as SMOTE and KMeans- SMOTE are used to generate synthetic samples for minority classes. These techniques help the model learn all sentiment categories equally and improve classification performance. In recent research, balanced datasets combined with transformer-based embeddings have shown high

accuracy in sentiment classification tasks. This approach is useful for analyzing tweets related to the Russia–Ukraine war, where negative or neutral tweets may be more common than positive ones.

Machine learning classifiers such as Logistic Regression, Random Forest, and Support Vector Machine are commonly used for sentiment analysis. Logistic Regression is simple and interpretable but cannot capture complex relationships between features. Random Forest improves accuracy by combining multiple decision trees, but it requires more computation and may be difficult to interpret. Support Vector Machine works well for high-dimensional data but is slow for very large datasets and cannot easily adapt to changing sentiment patterns. Because of these limitations, recent research suggests combining these classifiers with deep learning models to create hybrid systems that provide better accuracy and stability.

Class imbalance is also a common problem in sentiment analysis datasets. In many real-world situations, one sentiment category may have more samples than others. For example, during war discussions, negative sentiments may be more frequent than positive ones. If the model is trained on such data, it may become biased toward the majority class. To solve this problem, researchers use oversampling techniques such as SMOTE and KMeans-SMOTE to generate synthetic data for minority classes. These methods help in balancing the dataset and improving the performance of the classifier. Balanced datasets are very important for building reliable sentiment analysis systems.

Evaluation of sentiment analysis models is another important part of research. Different models may produce different results, so it is necessary to compare them using performance metrics. Accuracy, precision, recall, and F1-score are commonly used to measure the performance of classification models. In addition, confusion matrices and graphical charts are used to visualize the results. Comparative analysis helps researchers identify which model works best for a particular dataset. Many recent studies recommend using multiple classifiers and selecting the best one based on performance. This approach increases the reliability of the system and provides more accurate results.

Real-time sentiment analysis has also become an important research topic. Instead of analyzing data after collection, modern systems can process tweets continuously and provide instant results. These systems use efficient algorithms and powerful hardware to handle large amounts of data. Real-time sentiment monitoring is useful for understanding public reactions during live events such as wars, elections, and natural disasters. Governments, researchers, and

media organizations can use this information to study public opinion and make decisions based on current trends. Because the Russia–Ukraine war generated a large amount of online discussion, real-time sentiment analysis is very useful for understanding global reactions.

Based on the review of previous studies, it is clear that the combination of advanced preprocessing, transformer-based embeddings, data balancing techniques, and hybrid classification models can provide better performance for sentiment analysis. Many existing systems use only one model and do not compare different approaches. Therefore, there is a need for a comparative hybrid deep learning system that can evaluate multiple classifiers and identify the best model for analyzing Twitter sentiment. The proposed project focuses on this approach by combining RoBERTa embeddings, deep neural networks, KMeans-SMOTE balancing, and multiple machine learning classifiers to analyze sentiment related to the Russia–Ukraine war and to provide accurate and reliable results.

2.1 REVIEW OF RELATED WORK

The analysis of public sentiment from social media has been widely studied in the fields of Natural Language Processing (NLP), machine learning, and deep learning. Several approaches have been proposed to analyze opinions expressed in text data, especially from platforms like Twitter, which contain large amounts of real-time information. Researchers have developed different techniques ranging from rule-based sentiment analysis to advanced deep learning models to classify public opinion. This review discusses previous research and existing methodologies used for sentiment analysis, along with their advantages and limitations, and explains the need for a hybrid deep learning approach for analyzing Twitter sentiment related to the Russia-Ukraine war.

1. Traditional Content Moderation Approaches

Early sentiment analysis systems used rule-based methods and keyword dictionaries to classify text into positive, negative, or neutral categories. These methods were simple but could not understand the context of words and failed when dealing with informal text such as tweets. Social media data often contains slang, abbreviations, and sarcasm, which makes rule-based systems less accurate. Because of these limitations, researchers started using machine learning algorithms for better performance.

2. Machine Learning-Based Approaches

Machine learning algorithms such as Support Vector Machine (SVM), Logistic Regression, Naive Bayes, and Random Forest have been widely used for sentiment

classification. These methods use feature extraction techniques like Bag-of-Words and TF- IDF to convert text into numerical form. Machine learning models perform better than rule- based methods but still depend on manual feature selection and cannot fully understand the meaning of sentences. When applied to Twitter data related to the Russia-Ukraine war, these models sometimes give incorrect results due to noisy and unbalanced data.

3. Deep Learning-Based Approaches

Deep learning models such as Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), and Long Short-Term Memory (LSTM) networks improved sentiment analysis accuracy by learning complex patterns from text. LSTM and BiLSTM are useful for understanding sequential information in sentences, while CNN helps in extracting important features. Recent research also uses pretrained transformer models such as BERT and RoBERTa, which provide contextual embeddings and better understanding of sentence meaning. These models are effective for analyzing social media text but require high computational power and large datasets.

4. Recent Advances: Hybrid Models & Data Balancing

Recent studies focus on hybrid models that combine deep learning with machine learning classifiers. In this approach, deep neural networks extract features, and classifiers such as Logistic Regression, Random Forest, and SVM perform final prediction.Another challenge in sentiment analysis is class imbalance, where one sentiment category has more data than others. Techniques such as SMOTE and KMeans-SMOTE are used to balance the dataset and improve classification performance. Evaluation methods like accuracy, precision, recall, and confusion matrix are used to compare different models.

5. Comparison with the Proposed Approach

Existing sentiment analysis methods have limitations in handling noisy Twitter data and large datasets. Traditional machine learning models cannot understand context, while deep learning models alone may be computationally expensive. The proposed system improves previous work by using a hybrid deep learning approach for Twitter sentiment analysis on the Russia-Ukraine war. It uses preprocessing techniques, RoBERTa embeddings, KMeans- SMOTE balancing, deep neural networks, and multiple classifiers such as Logistic Regression, Random Forest, SVM, and Greedy Tree. This approach increases accuracy, handles imbalanced data, and provides better performance for large-scale sentiment analysis.

2.2 DEFINITION OF PROBLEM STATEMENT

The primary goal of this project is to develop a robust and scalable hybrid deep learning–based system capable of accurately analyzing and classifying public sentiment from Twitter data related to the Russia-Ukraine war. The system focuses on collecting large volumes of tweets, preprocessing noisy and unstructured social media text, extracting meaningful features using advanced pretrained language models, and classifying sentiments into positive, negative, or neutral categories with high accuracy. It aims to overcome the limitations of traditional sentiment analysis methods such as low accuracy, inability to understand contextual meaning, and poor performance on imbalanced datasets by using a combination of deep learning techniques, data balancing methods, and multiple machine learning classifiers. The proposed system also supports comparative analysis, efficient handling of large-scale data, and graphical visualization of results, making it suitable for real-time sentiment monitoring and helping researchers and analysts better understand global public opinion during major events such as the Russia-Ukraine war.

2.3 EXISTING SYSTEM

In the traditional approach, analysts manually read tweets related to the Russia-Ukraine war and categorize them into sentiment classes such as positive, negative, or neutral. Each tweet is carefully examined to understand its context, emotional tone, and intent. While this method can provide accurate results for individual tweets, it is extremely time-consuming and cannot handle the massive volume of tweets generated on social media daily. Human fatigue and subjective judgment may also lead to inconsistencies in labeling and interpretation. Another conventional approach relies on keyword-based sentiment analysis. Predefined sentiment lexicons assign fixed sentiment scores to certain words in tweets, such as “support,” “conflict,” or “peace.” The overall sentiment of a tweet is then determined based on the presence and frequency of these keywords. Although this method allows some level of automation, it cannot detect sarcasm, slang, context-dependent meanings, or newly emerging vocabulary on social media. Therefore, the results can often be misleading or incomplete. Traditional researchers also employ surveys or questionnaires to gauge public opinion about the Russia-Ukraine war. Participants provide their opinions in structured formats, which are later manually compiled and analyzed. While surveys provide more controlled data, they are limited in scope and fail to capture spontaneous reactions expressed on social media. Moreover,

survey responses may be influenced by social desirability bias, and the process is too slow to provide real-time sentiment insights.

Limitations of Existing System

Despite improvements in Twitter sentiment analysis, the existing system suffers from the following challenges:

- Inability to understand contextual meaning: Traditional machine learning and keyword- based methods classify tweets based on individual words and fail to understand the actual context of the sentence. Tweets containing sarcasm, irony, slang, or mixed emotions often lead to incorrect sentiment classification.
- High error rate in sentiment prediction: Manual and rule-based approaches cannot correctly differentiate between positive, negative, and neutral opinions in complex social media text. Similar words may express different emotions depending on context, which leads to false predictions and reduces accuracy.
- Poor performance on imbalanced and noisy data: Social media datasets often contain more tweets in one sentiment category than others, causing the model to become biased. In addition, tweets contain spelling errors, hashtags, and informal language, which makes it difficult for existing systems to generalize well on real-world data.
- Lack of scalability and adaptability: Keyword-based and simple machine learning methods cannot adapt to changing language trends on social media. As new words, abbreviations, and expressions appear frequently, existing systems fail to maintain accuracy and cannot provide reliable results for different datasets.

2.4 PROPOSED SYSTEM

The proposed system introduces a comparative hybrid deep learning approach for accurate Twitter sentiment analysis related to the Russia-Ukraine war. The system collects large volumes of tweets and performs preprocessing techniques such as tokenization, stopword removal, normalization, and lemmatization to clean the noisy social media text. After preprocessing, contextual feature extraction is performed using the RoBERTa pretrained language model to capture the semantic meaning of tweets. To handle the problem of imbalanced data, the KMeans-SMOTE technique is used to generate synthetic samples and balance the dataset. The extracted features are then passed through a deep neural network for

refinement and classified using multiple machine learning classifiers such as Logistic Regression, Random Forest, Support Vector Machine, and Greedy Tree. The system also performs comparative analysis using accuracy, precision, recall, F1-score, and confusion matrix to identify the best performing model. This approach improves classification accuracy, handles large-scale Twitter data efficiently, and provides reliable sentiment analysis for understanding public opinion during global events such as the Russia-Ukraine war.

Advantages of the Proposed System:

The proposed system significantly improves upon the existing approaches by addressing key limitations:

- **Better understanding of contextual meaning:** Using RoBERTa embeddings helps the system understand the meaning of tweets based on context, improving sentiment accuracy.
- **Higher accuracy with hybrid deep learning approach:** Combining deep learning with classifiers like Logistic Regression, Random Forest, and SVM gives better performance than single models.
- **Efficient handling of large-scale Twitter data:** The system can process large volumes of Twitter data quickly, making it suitable for real-time sentiment analysis.
- **Improved performance on imbalanced datasets:** KMeans-SMOTE balancing technique reduces bias and improves classification of all sentiment categories.
- **Scalable and adaptable to changing social media trends:** The model can handle noisy social media text and adapt to changing language trends on Twitter

2.5 OBJECTIVES

- To collect and preprocess large volumes of Twitter data related to the Russia-Ukraine war for sentiment analysis.
- To extract meaningful features from tweets using pretrained language models such as RoBERTa for better contextual understanding.
- To develop a hybrid deep learning system by combining deep neural networks with machine learning classifiers like Logistic Regression, Random Forest, SVM, and Greedy Tree.
- To handle imbalanced datasets using KMeans-SMOTE technique to improve classification accuracy.

- To compare different models using performance metrics such as accuracy, precision, recall, and F1-score to identify the best sentiment classification approach

2.6 HARDWARE & SOFTWARE REQUIREMENTS

2.6.1 HARDWARE REQUIREMENTS:

Hardware interfaces specifies the logical characteristics of each interface between the software product and the hardware components of the system. The following are some hardware requirements,

Processor	:	Intel Core i5
Memory	:	1TB
RAM	:	8GB

2.6.2 SOFTWARE REQUIREMENTS:

Software requirements specify the logical characteristics of each interface and the software components required to develop and run the proposed system. The following are the software requirements for the system:

Operating System	:	Windows 10
Language	:	Python
Database	:	Redis
Frontend	:	Tkinter

3. SYSTEM ARCHITECTURE & DESIGN

3. SYSTEM ARCHITECTURE & DESIGN

Project architecture refers to the structural framework and design of a project, encompassing its components, interactions, and overall organization. It provides a clear blueprint for development, ensuring efficiency, scalability, and alignment with project goals. Effective architecture guides the project's lifecycle, from planning to execution, enhancing collaboration and reducing complexity

3.1 PROJECT ARCHITECTURE

This project architecture shows the procedure followed for Comparative Hybrid Deep Learning Study of Twitter Sentiment on the Russia-Ukraine War

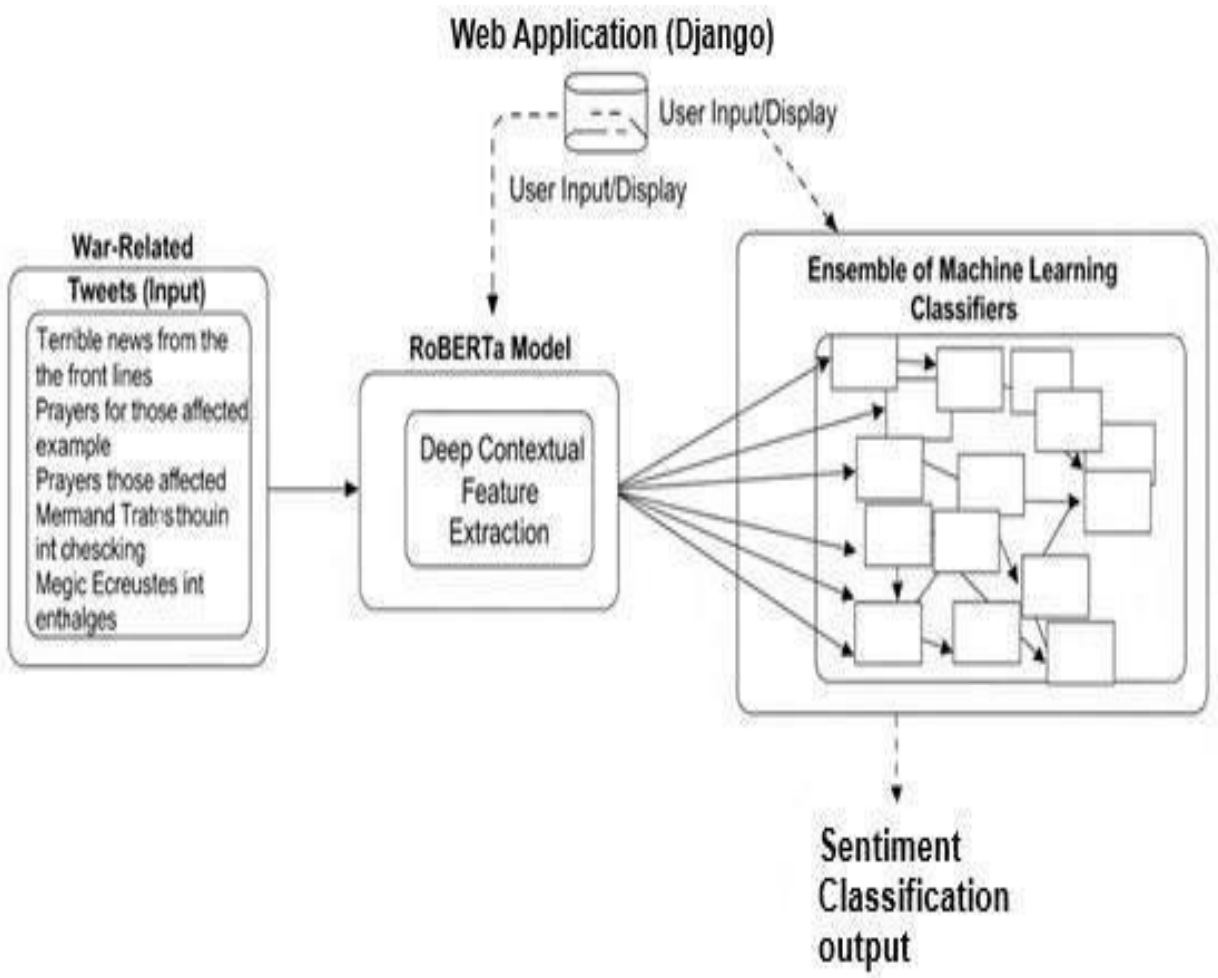

Figure 3.1: Project Architecture of A Comparative Hybrid Deep Learning Study of Twitter Sentiment on the Russia-Ukraine War

3.2 DESCRIPTION

- **Input Data** : The project uses Twitter data related to the Russia-Ukraine war. The dataset contains a large number of tweets collected using keywords, hashtags, and user discussions about the conflict. The data includes text content along with sentiment labels used for training and testing the model.
- **Reading Data** : The collected tweets are loaded into the system using data processing libraries. The text data is cleaned by removing URLs, special characters, stopwords, and unnecessary symbols to prepare it for analysis.
- **Feature Extraction** : RoBERTa, a pretrained transformer-based language model, is used to extract contextual embeddings from tweets. These embeddings represent the semantic meaning of the text and are used as input features for further processing.
- **Data Balancing** : KMeans-SMOTE technique is applied to balance the dataset by generating synthetic samples for minority sentiment classes. This helps the model learn all sentiment categories equally and improves classification performance.
- **Deep Feature** Learning : The extracted features are passed through a deep neural network to learn complex patterns and relationships in the text data. This step improves the quality of features before classification.
- **Classification Layer** : The refined features are given to multiple classifiers such as Logistic Regression, Random Forest, Support Vector Machine, and Greedy Tree to classify tweets into positive, negative, or neutral sentiment.
- **Training and Evaluation** : The model is trained using labeled tweet data and evaluated using performance metrics such as accuracy, precision, recall, F1-score, and confusion matrix to compare different classifiers and select the best model.
- **Visualization and Feedback** : The system displays results using graphs and charts to show sentiment distribution and model performance. The output can be used as feedback to improve the model and to better understand public opinion about the Russia-Ukraine war.

UNIFIED MODELING LANGUAGE(UML)

Unified Modeling Language (UML) is not just a static set of diagrams; it's a versatile toolset that supports multiple views and perspectives of a system. This versatility allows UML to be applied at different stages of the software development lifecycle, from initial requirements analysis and design to implementation, testing, and maintenance.

UML diagrams serve as a common visual language that bridges the gap between technical and non-technical stakeholders. They provide a means for software engineers to communicate complex system designs in a clear and concise manner, facilitating better understanding and collaboration among team members.

One of the key benefits of UML is its adaptability to different software development methodologies, such as waterfall, agile, and iterative approaches. UML diagrams can be used to model systems at various levels of abstraction, from high-level architectural views to detailed design specifications.

GOALS OF UML

1. Provide users a ready-to-use, expressive visual modeling language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.
3 .Be independent of particular programming languages and development processes.
4 .Provide a formal basis for understanding the modeling language.

3.3 USECASE DIAGRAM

A use case diagram in the Unified Modelling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

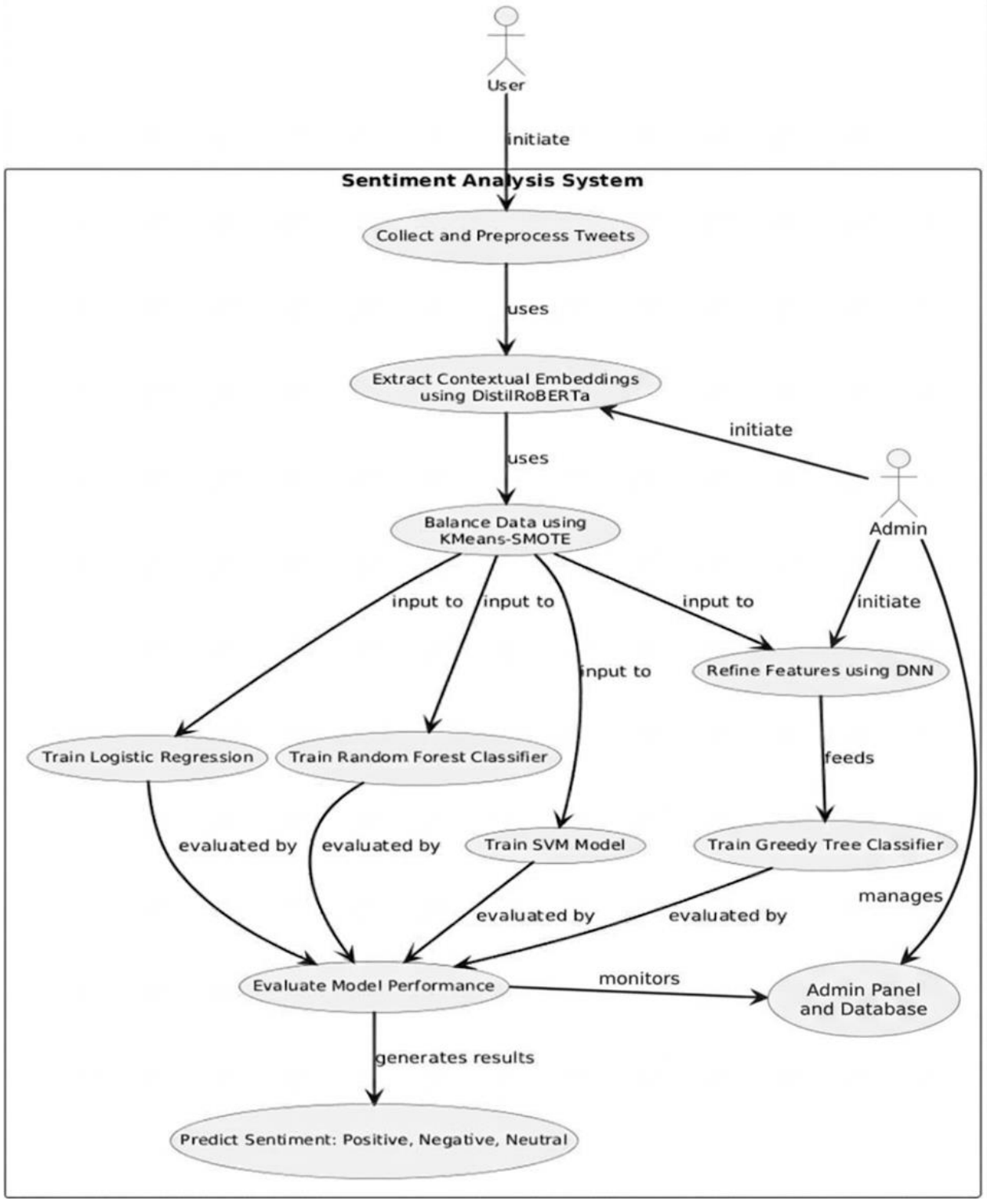

Figure 3.3 Use case Diagram of A Comparative Hybrid Deep Learning Study of Twitter Sentiment on the Russia Ukraine War.

3.4 CLASS DIAGRAM

In software engineering, a class diagram in the Unified Modelling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

A class diagram is a static structural diagram that represents the structure of a system by showing the system's classes, their attributes, methods, and relationships among them. It provides a high-level view of the system's architecture, focusing on the entities (classes) and their interactions. Class diagrams are typically used during the design phase of software development to visualize and organize the system's components and their relationships.

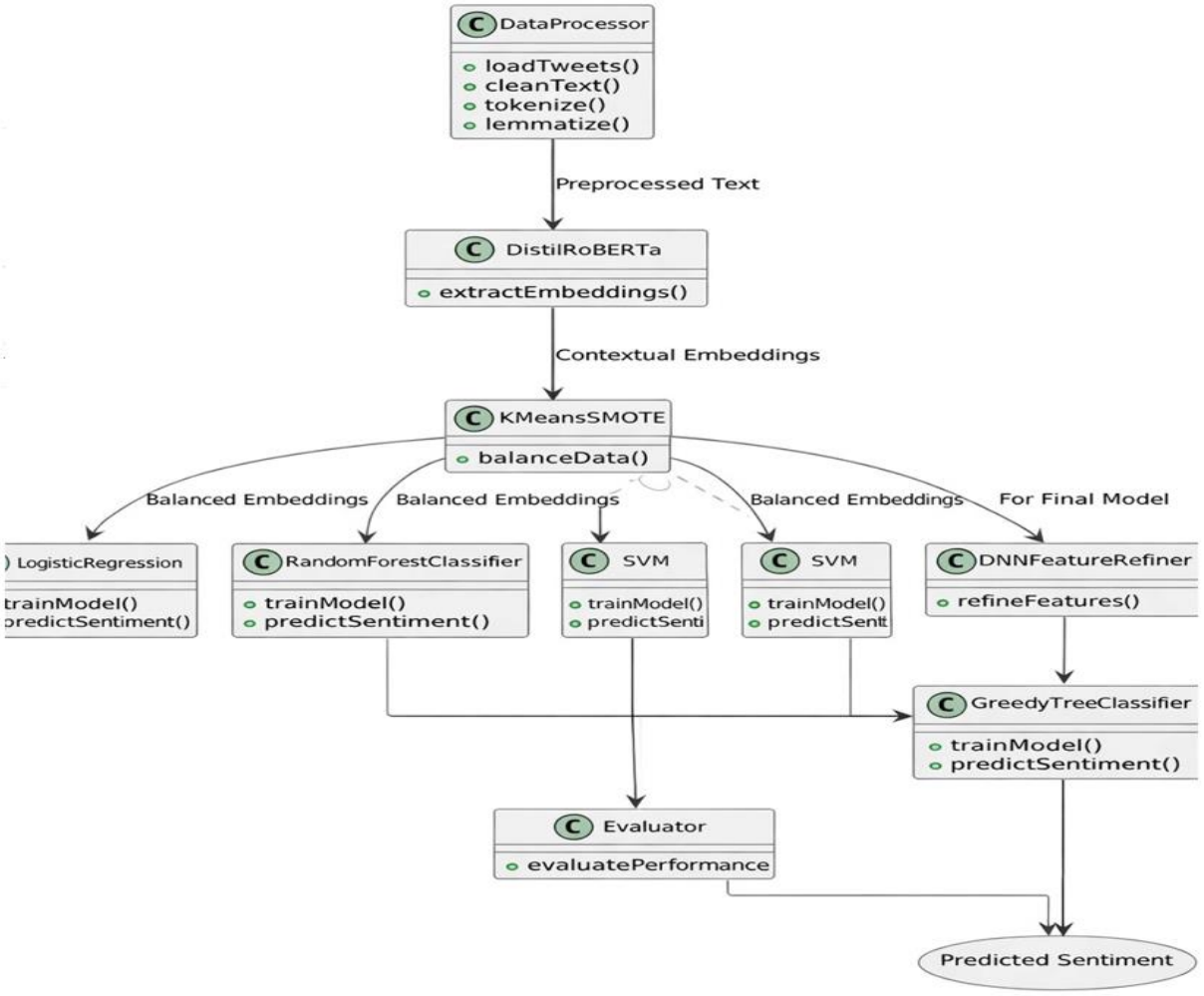

Figure 3.4 Class Diagram of A Comparative Hybrid Deep Learning Study of Twitter Sentiment on the Russia Ukraine War.

3.5 SEQUENCE DIAGRAM

A sequence diagram in Unified Modelling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams

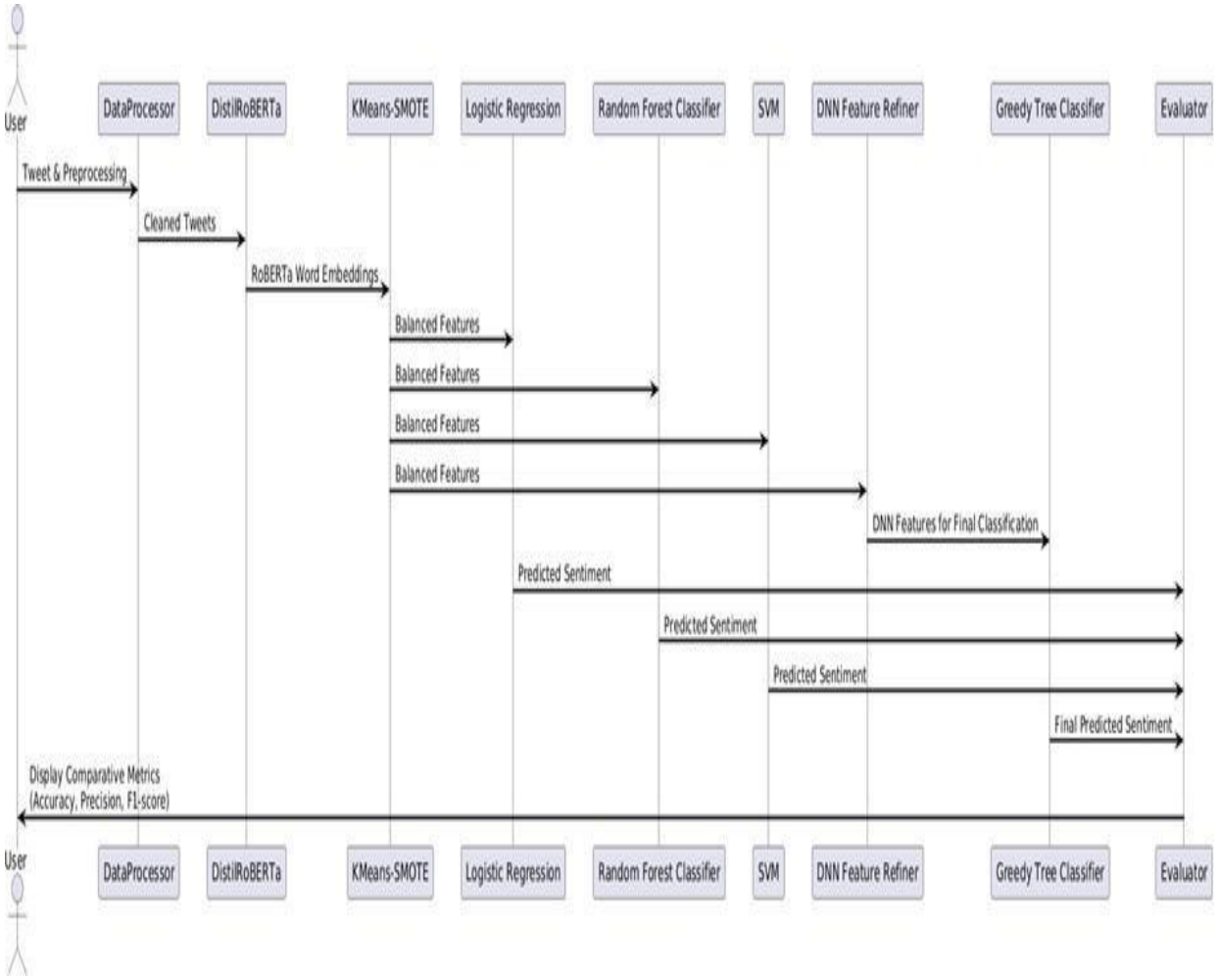

Figure 3.5 Sequence Diagram of A Comparative Hybrid Deep Learning Study of Twitter Sentiment on the Russia Ukraine War.

3.6 ACTIVITY DIAGRAM

An activity diagram visually represents the workflow of a system, showing the sequence of actions and decision points. In our project, it illustrates how a user uploads a coin image, the system preprocesses the image and extracts features using a Convolutional Neural Network (CNN) model, and then predicts the denomination of the coin. The predicted result is finally displayed to the user through the application interface.

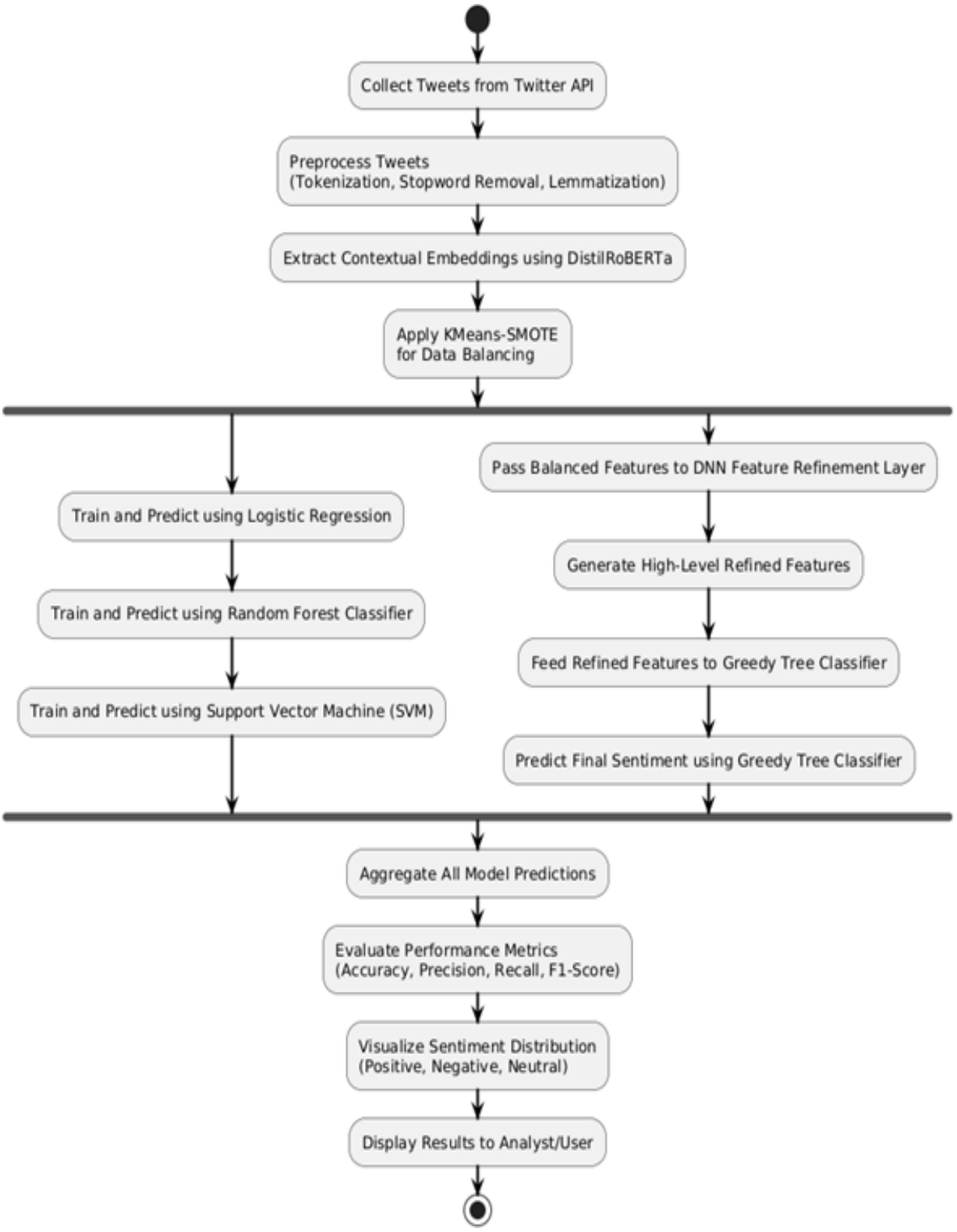

Figure 3.6 Activity Diagram of A Comparative Hybrid Deep Learning Study of Twitter Sentiment on the Russia Ukraine War

3.7 DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) is a graphical representation that illustrates how data flows within a system, showcasing its processes, data stores, and external entities. It is a vital tool in system analysis and design, helping stakeholders visualize the movement of information, identify inefficiencies, and optimize workflows.

A Data Flow Diagram comprises Four primary elements:

3.7.1 External Entities: Represent sources or destinations of data outside the system.

3.7.2 Processes: Indicate transformations or operations performed on data.

3.7.3 Data Flows: Depict the movement of data between components.

3.7.4 Data Stores: Represent where data is stored within the system.

These components are represented using standardized symbols, such as circles for processes, arrows for data flows, rectangles for external entities, and open-ended rectangles for data stores.

Benefits:

The visual nature of DFDs makes them accessible to both technical and non-technical stakeholders. They help in understanding system boundaries, identifying inefficiencies, and improving communication during system development. Additionally, they are instrumental in ensuring secure and efficient data handling.

Applications:

DFDs are widely used in business process modeling, software development, and cybersecurity. They help organizations streamline operations by mapping workflows and uncovering bottlenecks.

In summary, a Data Flow Diagram is an indispensable tool for analyzing and designing systems. Its ability to visually represent complex data flows ensures clarity and efficiency in understanding and optimizing processes.

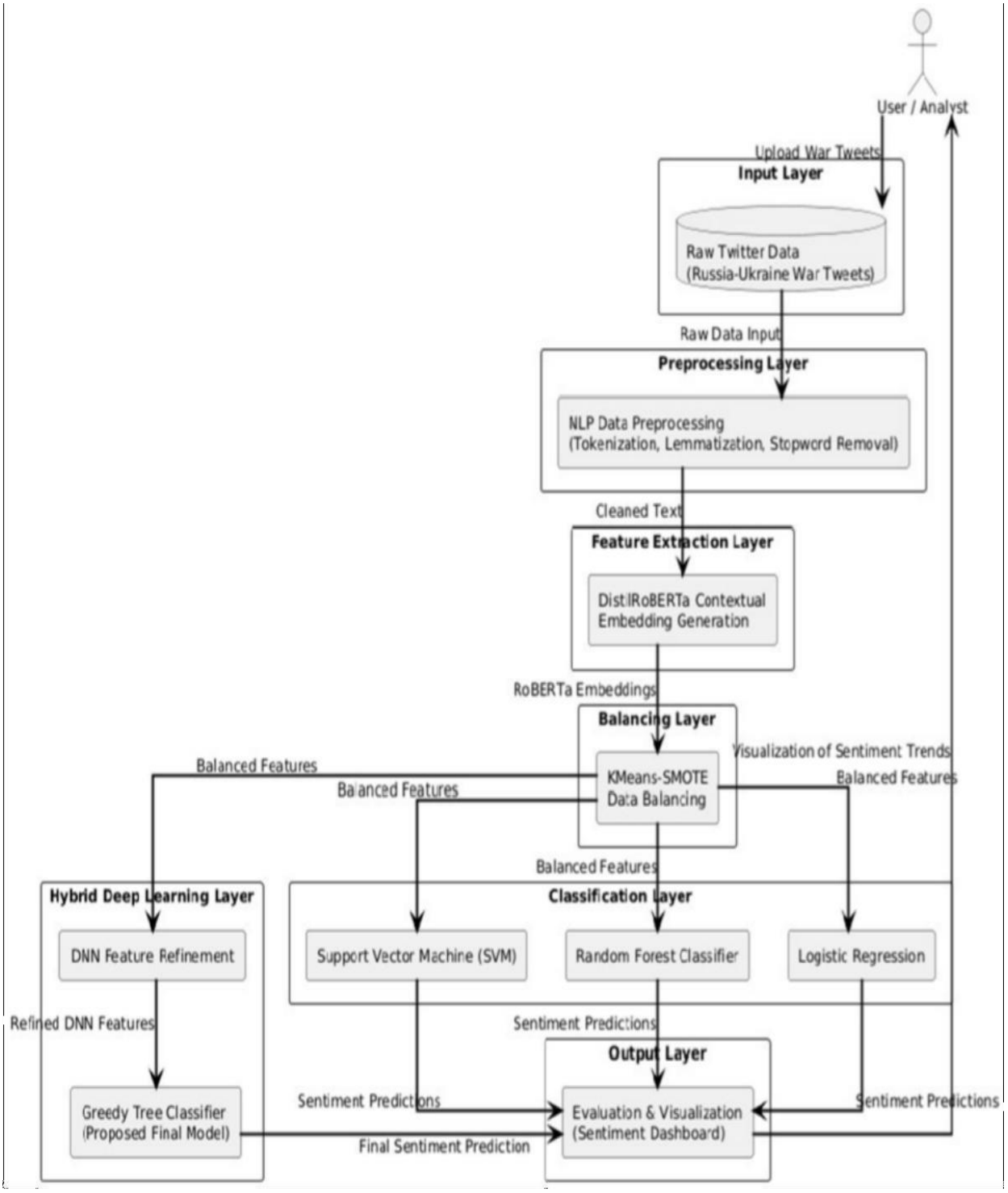

Figure 3.7 Dataflow Diagram of A Comparative Hybrid Deep Learning Study of Twitter Sentiment on the Russia Ukraine War.

4. IMPLEMENTATION

4. IMPLEMENTATION

The implementation phase of a project involves executing the planned strategies and tasks. It requires meticulous coordination, resource allocation, and monitoring to ensure that objectives are met efficiently. Effective implementation is crucial for achieving project goals and delivering expected outcomes within the set timeline and budget constraints.

4.1 ALGORITHMS USED

Transformer Based Models for Feature Extraction

Transformer-based language models are widely used for text classification because they can capture contextual meaning from sentences. In this project, the RoBERTa pretrained model is used to extract contextual embeddings from tweets. Unlike traditional feature extraction methods such as TF-IDF or Bag-of-Words, RoBERTa understands the relationship between words in a sentence and generates meaningful representations. These embeddings help the system identify whether a tweet expresses positive, negative, or neutral sentiment related to the Russia-Ukraine war. The extracted features are then passed to deep learning and machine learning classifiers for further analysis.

Advantages of Transformer-Based Models:

- Provides better contextual understanding of text.
- Works well for complex and noisy social media data.
- Improves accuracy compared to traditional methods.
- Pretrained on large datasets, so feature quality is high.

Disadvantages of Transformer -Based Models:

- Requires high computational power.
- Large model size increases memory usage.
- Training and fine-tuning take more time.
- Not efficient without GPU support.

Deep Neural Network for Feature Learning

After extracting embeddings using RoBERTa, a Deep Neural Network (DNN) is used to learn complex relationships in the data. The neural network refines the extracted features and improves the quality of input before classification. This step helps the system understand

patterns in tweets, including emotional expressions, sarcasm, and mixed opinions. Deep learning models are effective for large datasets and improve overall prediction performance.

Advantages of Deep Neural Network:

- Learns complex patterns from data.
- Improves feature quality.
- Works well with large datasets.
- Increases classification accuracy.

Disadvantages of Deep Neural Network:

- Requires high computational resources.
- Training takes more time.
- Risk of overfitting.
- Difficult to interpret results.

1. KMeans-SMOTE for Data Balancing

In sentiment datasets, some classes may contain more tweets than others. This causes the model to become biased toward the majority class. To solve this problem, KMeans-SMOTE is used to balance the dataset by generating synthetic samples for minority classes. This method improves the ability of the classifier to learn all sentiment categories equally and increases the overall performance of the system.

- Improves accuracy on imbalanced datasets.
- Reduces bias toward majority class.
- Helps model learn all categories equally.

2. Random Forest for Improved Classification

Random Forest is used as an ensemble classifier to improve sentiment prediction accuracy. It works by combining multiple decision trees and selecting the best result from all trees. In this project, features extracted from RoBERTa and refined by the neural network are given to the Random Forest classifier for final sentiment prediction. This improves stability and reduces overfitting.

- High accuracy.
- Works well for large datasets.
- Reduces overfitting.
- Handles nonlinear data.

3. Support Vector Machine (SVM) for Baseline Classification

Support Vector Machine is used as a baseline classifier in this project. It separates sentiment classes by finding the best boundary between them. SVM works well for text classification with high-dimensional data. However, its performance is lower compared to hybrid deep learning models because it cannot fully understand complex contextual relationships in tweets.

- Good for text classification.
- Works well with high-dimensional data.
- High accuracy for small datasets.

4. Logistic Regression for Sentiment Prediction

Logistic Regression is used as a simple classification algorithm to predict tweet sentiment. It calculates the probability of each class and selects the most likely sentiment. This algorithm is fast and easy to implement, but it cannot capture complex relationships like deep learning models.

- Requires less memory.
- Works well for basic classification.

5. Greedy Tree Classifier:

Greedy Tree is a decision-tree-based algorithm that selects the best feature at each step to classify sentiment. It is used in this project to compare performance with other classifiers. The algorithm is simple and fast but may overfit when the dataset is large.

- Easy to understand.
- Fast prediction.
- Works well for classification tasks.
- Requires less preprocessing.

Now-a-days social media platforms such as Twitter are widely used by people all over the world to express their opinions, emotions, and reactions about real-world events. During international conflicts like the Russia-Ukraine war, millions of users post tweets every day discussing political decisions, military actions, humanitarian issues, and global responses. These tweets contain valuable information that reflects public sentiment, but analyzing such a

huge amount of data manually is extremely difficult and time-consuming. Moreover, social media text often includes sarcasm, slang, abbreviations, and mixed opinions, which makes sentiment understanding more complicated. Therefore, there is a need for an automated system that can accurately analyze public opinion from large-scale Twitter data.

Traditional sentiment analysis methods mainly rely on manual inspection, keyword matching, or simple machine learning models. These methods are not efficient for handling millions of tweets and often fail to understand the real meaning of the text because they cannot capture context. For example, a tweet may contain positive words but express a negative opinion depending on the situation. In the case of the Russia-Ukraine war, tweets may include emotional reactions, political sarcasm, or neutral news statements, which require advanced techniques for correct classification. Hence, a more powerful and intelligent approach is required to perform sentiment analysis with better accuracy and speed.

To overcome these limitations, this project proposes a hybrid deep learning approach for analyzing Twitter sentiment related to the Russia-Ukraine war. In this work, tweets are collected using keywords and hashtags related to the conflict. After collecting the data, preprocessing steps such as removing special characters, links, stop words, and unnecessary symbols are applied to clean the text. This step is very important because social media data is usually noisy and unstructured. After preprocessing, the cleaned tweets are converted into numerical representations using a pretrained language model called RoBERTa, which is capable of understanding the contextual meaning of words in a sentence.

RoBERTa is a transformer-based deep learning model that provides better text representations compared to traditional feature extraction methods like TF-IDF or Bag-of- Words. It generates embeddings that capture semantic relationships between words, making it easier for the system to understand the actual sentiment of the tweet. These extracted features are then passed to different machine learning classifiers such as Logistic Regression, Random Forest, Support Vector Machine, and Greedy Tree. Using multiple classifiers helps in comparing performance and selecting the best model for sentiment prediction.

In the proposed work, a comparative hybrid architecture is developed to improve the overall accuracy of sentiment classification. Instead of using a single algorithm, the system

combines deep learning feature extraction with multiple classification models. This hybrid approach helps in capturing both contextual meaning and statistical patterns in the data. In addition, the dataset used in this project may contain imbalanced sentiment classes, where one sentiment type appears more frequently than others. To solve this problem, a KMeans-SMOTE data balancing technique is applied to generate synthetic samples for minority classes, which improves the performance of the classifiers.

Different algorithms are trained and tested on the same dataset to evaluate their performance using metrics such as accuracy, precision, recall, and F1-score. Experimental results show that the hybrid deep learning approach provides better accuracy compared to traditional machine learning methods.

4.1 Dataset of Twitter Related to Russia Ukraine War

Word Cloud of Top 100 Terms This word cloud visualizes the in font size proportional to frequency the most common terms in the dataset. Dominant words include “war,” “Russia,” “Ukraine,” “invasion,” “Putin,” “peace,” and “NATO,” reflecting core themes. However, large numerical strings (e.g., “1.63036e+18”) appear prominently, indicating metadata or ID leakage. These artifacts suggest incomplete text cleaning and potential noise in downstream NLP pipelines. The visualization effectively highlights geopolitical focus but exposes preprocessing flaws.

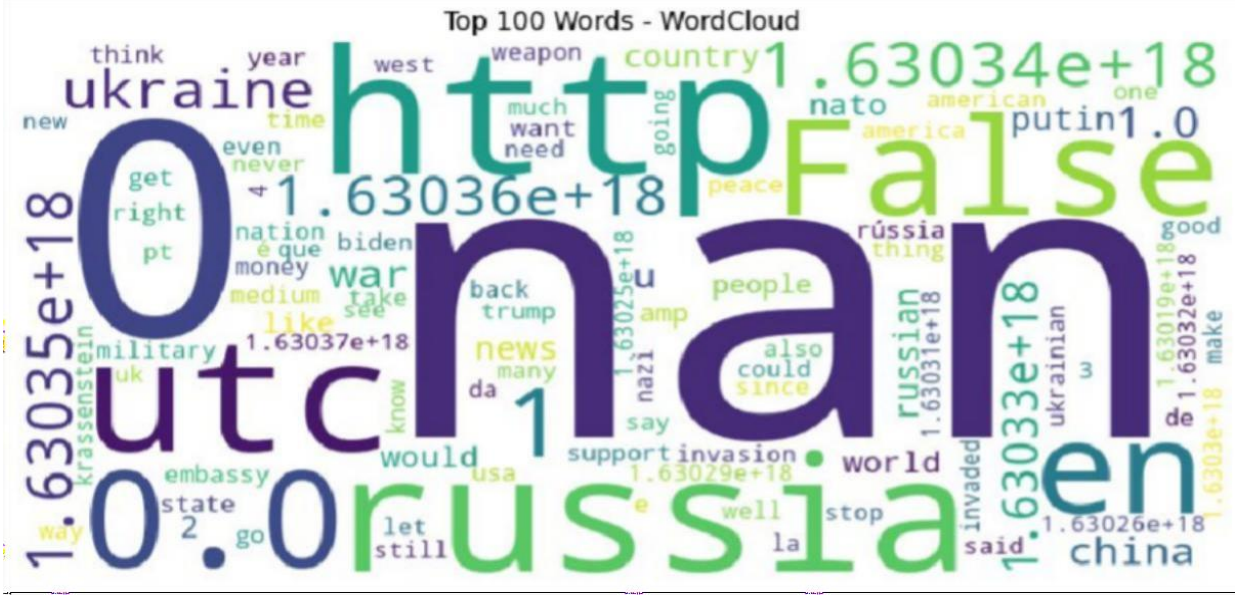

Figure 4.1 : Word Cloud of the Top 100 Most Frequent Terms in the Russia-Ukraine War Twitter Dataset.

To implement this project we have designed the following modules:

1) **Upload Twitter Sentiment Dataset:** Using this module, we upload the Twitter dataset related to the Russia-Ukraine war into the application. The dataset contains tweets collected using keywords, hashtags, and user discussions about the conflict. Each tweet includes text data along with sentiment labels used for training and testing the model.

2) **Dataset Preprocessing:** Using this module, the uploaded tweets are cleaned and prepared for analysis. Preprocessing includes removing URLs, special characters, punctuation marks, and stop words. All tweets are converted into a proper format, tokenized, and normalized so that the text can be processed by the deep learning model. The dataset is also shuffled to avoid bias during training.

3) **Run Proposed RoBERTa-Hybrid Algorithm:** Using this module, the dataset is divided into training and testing data where 80% of the data is used for training and 20% is used for testing. The training data is given to the pretrained RoBERTa model to extract contextual embeddings from tweets. These features are then passed to different classifiers such as Logistic Regression, Random Forest, Support Vector Machine, and Greedy Tree to train the hybrid model. The trained model is tested on the remaining data to calculate prediction accuracy.

4) **Run RoBERTa-SVM Algorithm:** In this module, features extracted using the RoBERTa model are given to the Support Vector Machine classifier. The SVM model is trained using the training dataset and then tested using the testing dataset to calculate accuracy and performance.

5) **Run RoBERTa-Random Forest Algorithm:** In this module, the features generated by RoBERTa are used to train the Random Forest classifier. The model learns patterns from the training data and predicts sentiment on the test data. The prediction results are used to measure accuracy, precision, recall, and F1-score.

6) **Comparison Graph:** Using this module, the system generates graphs to compare the performance of different models such as RoBERTa-Logistic Regression, RoBERTa-SVM, RoBERTa-Random Forest, and RoBERTa-Greedy Tree. The comparison graph helps to identify which algorithm provides better accuracy for sentiment analysis.

7) **Sentiment Prediction from New Tweets:** Using this module, the user can enter new tweets related to the Russia-Ukraine war. The system processes the tweet using the trained hybrid model and predicts the sentiment as Positive, Negative, or Neutral. This module demonstrates how the system can be used for real-time sentiment analysis.

4.2 SAMPLE CODE

```
from django.shortcuts import render, redirect
from django.views.decorators.csrf import csrf_exempt
from django.contrib import messages
from django.conf import settings
import pymysql
import pandas as pd
from joblib import load
# ================================
# Core Python Libraries
# ================================
import os
import pickle
import joblib
import numpy as np
import pandas as pd
from collections import Counter
from scipy.special import expit # sigmoid
from tqdm import tqdm

# ================================
# Data Visualization
# ================================
import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud
# ================================
# NLP: NLTK & Gensim
# ================================
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
```

```
from nltk import pos_tag
from nltk.util import ngrams

# ================================
# Scikit-learn: Preprocessing, Models, Metrics
# ================================
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

# ================================
# TensorFlow / Keras
# ================================
/import tensorflow as tf
from tensorflow.keras.models import Sequential, Model, load_model
from tensorflow.keras.layers import Dense, Input, Embedding, Conv1D,
GlobalMaxPooling1D, LSTM, Dropout
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical

# ================================
# Transformers (Hugging Face)
# ================================
from transformers import (
    AutoTokenizer, AutoModel,
    RobertaTokenizer, RobertaModel,
    BertTokenizer, BertForSequenceClassification,
    XLNetTokenizer, XLNetForSequenceClassification
)
import torch
from torch.optim import AdamW

# ================================
```

```
# Custom Modules
# ==============================

MODEL_DIR = r"warapp\model"

os.makedirs(MODEL_DIR, exist_ok=True)

import nltk

# Tokenizer
nltk.download('punkt', quiet=True)
# Stopwords
nltk.download('stopwords', quiet=True)
# WordNet for lemmatizer
nltk.download('wordnet', quiet=True)
#  POS tagging for better lemmatization
nltk.download('averaged_perceptron_tagger', quiet=True)

def ensure_single_admin():
    try:
        con = pymysql.connect(
            host='127.0.0.1',
            port=3306,
            user='root',
            password='root',
            database='mydb'
)
        with con:
            cur = con.cursor()
            cur.execute("SELECT id FROM emp_details3 WHERE role='admin'")
            admin = cur.fetchone()
            if not admin:
                cur.execute("""
```

```
            INSERT INTO emp_details3
            (username, email, password, role, approved, address, mobile)
            VALUES (%s, %s, %s, %s, %s, %s, %s)
        """, (
            "admin",
            "admin@gmail.com",
            "admin",
            "admin",
            1,
            "hyderabad",
            "1234567890"
        ))
        con.commit()

    except Exception as e:
        print("Error ensuring admin:", e)

def index(request):
    return render(request, 'index.html')

def user_page(request):
    user = request.session.get('user')
    if not user:
        return redirect('login')
    return render(request, 'user.html', {'user': user})
@csrf_exempt
def approve_user(request, username):
    user = request.session.get('user')
    if not user or user.get('role') != 'admin':
        return redirect('login')
    try:
        con = pymysql.connect(
            host='127.0.0.1',
```

```
            port=3306,
            user='root',
            password='root',
            database='mydb'
        )
        with con:
            cur = con.cursor()
              cur.execute("UPDATE emp_details3 SET approved = 1 WHERE username = %s",
(username,))
            con.commit()
    except Exception as e:
        print("Error approving user:", e)
    return redirect('admin_page')
def admin_page(request):
    user = request.session.get('user')
    if not user:
        return redirect('login')
    if user.get('role') != 'admin':
        return redirect('user_page')
    users_list = []
    try:
        con = pymysql.connect(
            host='127.0.0.1',
            port=3306,
            user='root',
            password='root',
            database='mydb'
        )
        with con:
            cur = con.cursor(pymysql.cursors.DictCursor)
            cur.execute("SELECT * FROM emp_details3 WHERE role='user'")
            users_list = cur.fetchall()
```

```
    except Exception as e:
        return render(request, 'admin.html', {
            'user': user,
            'error': f'Database error: {str(e)}'
        })

    return render(request, 'admin.html', {'user': user, 'users_list': users_list})

def register_view(request):
    ensure_single_admin()
    message = None
    if request.method == 'POST':
        username = request.POST.get('username')
        email = request.POST.get('email')
        password = request.POST.get('password')
        confirm_password = request.POST.get('confirm_password')
        mobile = request.POST.get('mobile')
        address = request.POST.get('address')
        role = "user"
        if password != confirm_password:
            message = {'error': 'Passwords do not match'}
else:
            try:
                con = pymysql.connect(
                    host='127.0.0.1',
                    port=3306,
                    user='root',
                    password='root',
                    database='mydb'
                )
                with con:
                    cur = con.cursor()
```

```
                cur.execute("SELECT username FROM emp_details3 WHERE username=%s",
(username,))
                if cur.fetchone():
                    message = {'error': 'Username already exists'}
                else:
                cur.execute("SELECT email FROM emp_details3 WHERE email=%s", (email,))
                    if cur.fetchone():
                        message = {'error': 'Email already exists'}
                        cur.execute("SELECT mobile FROM emp_details3 WHERE mobile=%s",
(mobile,))
                    if cur.fetchone():
                        message = {'error': 'mobile already exists'}
                    else:
                        cur.execute("""
                            INSERT INTO emp_details3
                            (username, email, password, role, approved, mobile, address)
                            VALUES (%s, %s, %s, %s, %s, %s, %s)
                        """, (username, email, password, role, 0, mobile, address))
                        con.commit()
                            message = {'success': 'Account created successfully! Awaiting admin
approval.'}

        except Exception as e:
            message = {'error': f'Database error: {str(e)}'}
    return render(request, 'register.html', message)

def login_view(request):
    context = {}
    if request.method == 'POST':
        username = request.POST.get('username')
        password = request.POST.get('password')
        try:
            con = pymysql.connect(
                host='127.0.0.1',
```

```
            port=3306,
            user='root',
            password='root',
            database='mydb'
        )
        with con:
            cur = con.cursor(pymysql.cursors.DictCursor)
            cur.execute(
                "SELECT * FROM emp_details3 WHERE username=%s AND password=%s",
                (username, password)
            )
            user = cur.fetchone()
            if user:
                if not user.get('approved'):
                    context['error'] = 'Your account is awaiting admin approval.'
                else:
                    request.session['user'] = user
                    role = user['role']
                    if role == 'admin':
                        return redirect('admin_page')
                    elif role == 'user':
                        return redirect('user_page')
                    else:
                        context['error'] = 'Invalid role assigned to this user.'
            else:
                context['error'] = 'Invalid login credentials'

    except Exception as e:
        context['error'] = f'Database error: {str(e)}'
    return render(request, 'login.html', context)

def preprocess_data(df, save_path=None, target_cols=None):
    global label_encoders
```

```
    label_encoders = {}  # dictionary to hold encoders for each target column
    if save_path and os.path.exists(save_path):
        print(f"Loading existing preprocessed file: {save_path}")
        df = pd.read_csv(save_path)
    else:
          print("Preprocessing data" + (f" and saving to: {save_path}" if save_path  else " (no
saving)"))
        lemmatizer = WordNetLemmatizer()
        stop_words = set(stopwords.words('english'))
        def clean_text(text):
            text = str(text).lower()
            tokens = word_tokenize(text)
           tokens = [lemmatizer.lemmatize(t) for t in tokens if t.isalnum() and t not in stop_words]
            return ' '.join(tokens)
        # Separate target columns
        target_df = None
        if target_cols:
            existing_targets = [col for col in target_cols if col in df.columns]
            target_df = df[existing_targets].copy()
            df = df.drop(columns=existing_targets)

        # Process text columns
        text_columns = df.select_dtypes(include='object').columns
        for col in text_columns:
            df[f'processed_{col}'] = df[col].apply(clean_text)

        # Drop original text columns
        df.drop(columns=text_columns, inplace=True)

        # Reattach target columns
        if target_df is not None:
            for col in target_df.columns:
                df[col] = target_df[col]
```

```
        # Save only if path is specified
        if save_path:
            df.to_csv(save_path, index=False)
    # Select processed and numerical columns
    processed_text_cols = [col for col in df.columns if col.startswith('processed_')]
    non_text_cols = [col for col in df.columns if col not in processed_text_cols + (target_cols if
target_cols else [])]

    # Join processed text columns into one string
    X_text = df[processed_text_cols].astype(str).agg(' '.join, axis=1)

# Combine with numerical columns if any
    X_numeric = df[non_text_cols].values if non_text_cols else None
    if X_numeric is not None and len(X_numeric) > 0:
        X = [f"{text} {' '.join(map(str, numeric))}" for text, numeric in zip(X_text, X_numeric)]
    else:
        X = X_text.tolist()

    # Encode multiple target columns
    Y_dict = {}
    if target_cols:
        for col in target_cols:
            if col in df.columns:
                le = LabelEncoder()
                Y_dict[col] = le.fit_transform(df[col])
                label_encoders[col] = le

    return X, Y_dict

def roberta_feature_extraction(texts, model_name='roberta-base', batch_size=32,
pooling='mean'):
    """Extract RoBERTa features from texts with tqdm progress bar."""
```

```
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModel.from_pretrained(model_name)
    model.eval()

    all_embeddings = []

for i in tqdm(range(0, len(texts), batch_size), desc="Extracting RoBERTa embeddings"):
        batch_texts = texts[i:i+batch_size]
                encoded_input = tokenizer(batch_texts, padding=True, truncation=True,
return_tensors='pt')
with torch.no_grad():
            model_output = model(**encoded_input)

                token_embeddings = model_output.last_hidden_state  # [batch_size, seq_len,
hidden_dim]
        attention_mask = encoded_input['attention_mask']
                input_mask_expanded = attention_mask.unsqueeze(-
1).expand(token_embeddings.size()).float()

        if pooling == 'mean':
            sum_embeddings = torch.sum(token_embeddings * input_mask_expanded, dim=1)
            sum_mask = input_mask_expanded.sum(dim=1)
            embeddings = sum_embeddings / sum_mask
        elif pooling == 'cls':

            embeddings = token_embeddings[:, 0, :] # CLS token
        else:
            raise ValueError("Pooling must be 'mean' or 'cls'")

        all_embeddings.append(embeddings.cpu().numpy())

    X = np.vstack(all_embeddings)
    return X, model
```

```
def feature_extraction(X_text, method='RoBERTa_word_embeddings',
model_dir=r'warapp\model', is_train=True):
    x_file = os.path.join(model_dir, f'X_{method}.pkl')


    print(f"[INFO] Feature extraction method: {method}, Train mode: {is_train}")
    model_name = 'distilroberta-base'  # Lightweight alternatives: 'roberta-small', 'distilroberta-
base'

if is_train:
        if os.path.exists(x_file):
            print(f"[INFO] Loading cached RoBERTa features from {x_file}")
            X = joblib.load(x_file)
        else:
            print("[INFO] Computing RoBERTa features...")
                    X, model = roberta_feature_extraction(X_text, model_name=model_name,
pooling='mean')
            os.makedirs(model_dir, exist_ok=True)
            joblib.dump(X, x_file)
    else:
        print("[INFO] Performing RoBERTa feature extraction for testing...")
                X, model = roberta_feature_extraction(X_text, model_name=model_name,
pooling='mean')
    return X

def test_DNN_feature_extraction(features_test, model_base_name="Ditil RoBERT-WE
DNN", model_dir=r'warapp\model'):
    feature_outputs = {}
    targets = ["Sentiment"]
    for target in targets:
        model_filename = f"{model_base_name}_{target}_dense_model.h5"
        model_path = os.path.join(model_dir, model_filename)
```

```
        if not os.path.exists(model_path):
            print(f"❌ Model not found: {model_path}")
            continue
        print(f"✅ Loading model for {target}: {model_path}")
    try:
            feature_extractor = Model(inputs=model.input,
                                      outputs=model.get_layer("feature_layer").output)
        except:
                raise ValueError(f"Model does not contain a layer named 'feature_layer': {model_path}")
        print(f"🔄 Extracting features for {target}...")
        extracted_features = feature_extractor.predict(features_test)
        feature_outputs[target] = extracted_features
    return feature_outputs

Final_models = {}
model_path = os.path.join(MODEL_DIR, "Distil RoBERT-WE DNN_Sentiment_GreedyTree_model.pkl")
if os.path.exists(model_path):
    print(f"Loading sentiment model from: {model_path}")
    Final_models["Sentiment"] = joblib.load(model_path)
else:
    print(" Sentiment model file not found. Please train and save it first.")

def prediction_page(request):
    prediction_table = None
    uploaded_filename = None
    if request.method == 'POST' and request.FILES.get('file'):
        file = request.FILES['file']
        uploaded_filename = file.name
        df_test1 = pd.read_csv(file)
        df_result = df_test1.copy()
```

```
        #Preprocess uploaded file
        df_test, _ = preprocess_data(df_test1)
features_test = feature_extraction(df_test,method='Distil RoBERT with Word Embeddings',is_train=None)
        features_test = feature_extraction(df_test,method='RoBERT-WE DNN',is_train=None)
        feature_outputs_dict = test_DNN_feature_extraction(features_test)
        classes = [ "Negative","Neutral", "Positive"]

# Loop over each target
        for target in ['Sentiment']:
            target_features = feature_outputs_dict[target]
            y_pred = Final_models[target].predict(target_features)
            mapped_labels = [classes[int(pred)] for pred in y_pred]
            df_result[f'Predicted_{target.capitalize()}'] = mapped_labels
            df_result.insert(0, "Sl.No", range(0, len(df_result)))

        #  Display prediction table
        prediction_table = df_result.to_html(
            classes='table table-bordered table-striped table-hover',
            index=False
        )
                messages.success(request, f"Predictions generated successfully for {uploaded_filename}")
    return render(
        request,
        'prediction.html',
        {
            'prediction_table': prediction_table,
            'uploaded_filename': uploaded_filename
        }
)
```

5.RESULTS

&

DISCUSSION

5. RESULTS & DISCUSSION

The following screenshots showcase the results of our project, highlighting key features and functionalities. These visual representations provide a clear overview of how the system performs under various conditions, demonstrating its effectiveness and user interface. The screenshots serve as a visual aid to support the project's technical and operational achievements.

5.1 Home Page :

Illustrates the main landing interface of the sentiment analysis system, where users are introduced to the platform's purpose and navigation options. It presents an accessible layout that guides users towards registration, login, or exploring features. The interface shows a clean, user-friendly environment designed to enhance usability. It highlights the system's focus on analyzing public sentiment about the Russia–Ukraine conflict..

Figure 5.1 : Home page of sentiment analysis of Russia Ukraine war.

5.2 Registration Page :

It shows the user registration module where new users can securely create an account to access the system's analytical features. The interface illustrates a structured form capturing essential credentials to ensure authenticated usage. It presents validation mechanisms that enhance security and data integrity. This page depicts the entry point for users to begin interacting with the sentiment analysis platform.

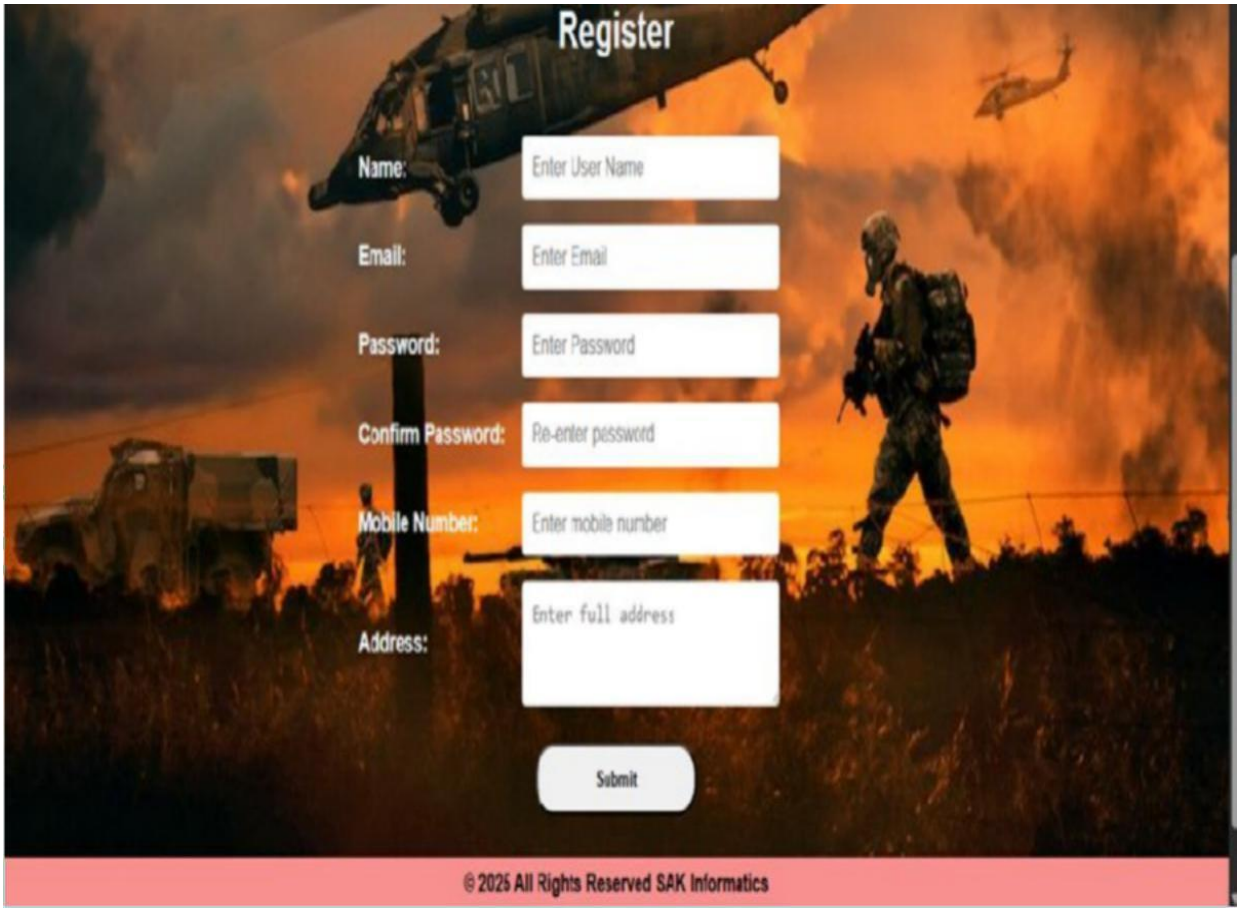

Figure 5.2 : Registration page of sentiment analysis of Russia Ukraine war.

5.3 Login Page :

It presents the login interface that allows existing users to access the system through verified credentials. It illustrates a streamlined login form designed for quick and secure authentication. The layout shows essential input fields supported by backend verification processes. This page demonstrates the controlled access mechanism ensuring that only authorized users can perform sentiment analysis operations.

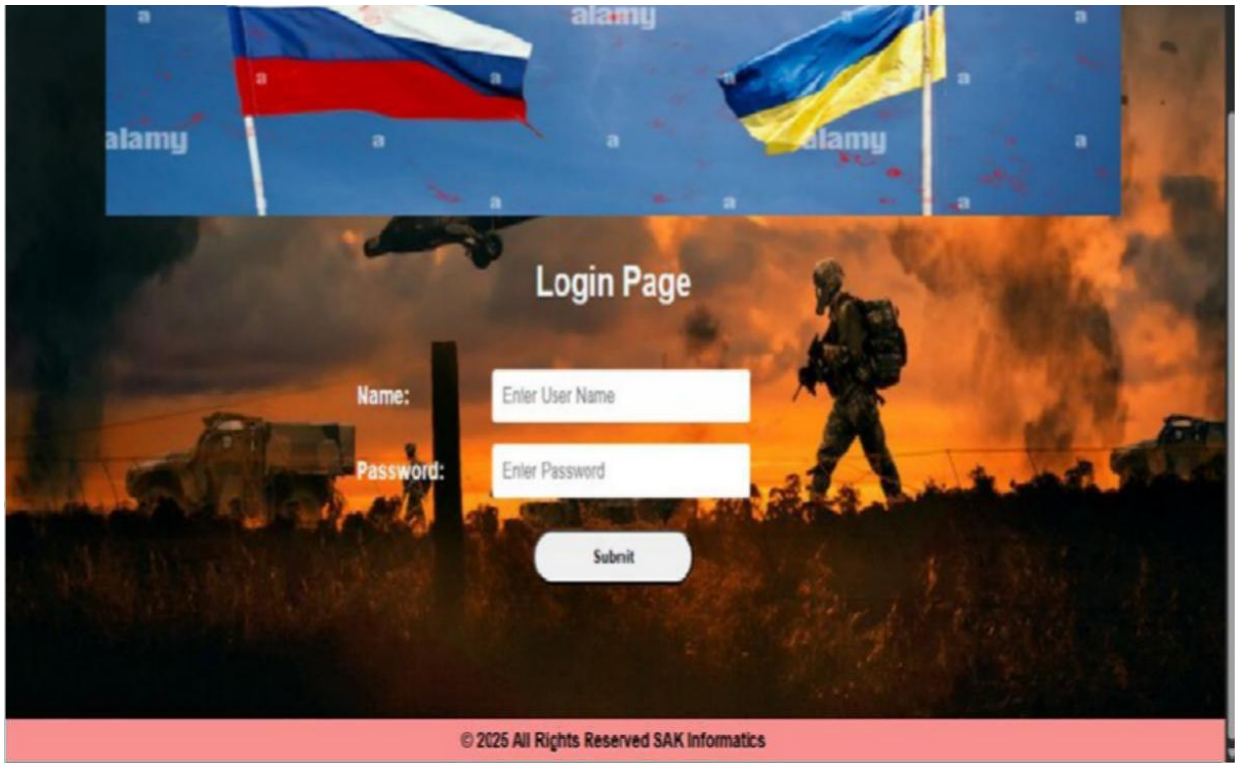

Figure 5.3 : Login page of sentiment analysis of Russia Ukraine war.

5.4 Statistical Representation :

Distribution of Sentiment Classes The bar chart reveals a marked class imbalance in the Russia-Ukraine War Twitter dataset. Negative (Class 0) and Neutral (Class 1) sentiments each account for over 3,500 tweets, while Positive (Class 2) tweets number approximately 2,500, indicating a lower prevalence of optimistic discourse. This skewed distribution underscores the predominantly critical or neutral tone in public reactions to the conflict. Such imbalance necessitates advanced oversampling techniques like KMeans-SMOTE to ensure robust model training.

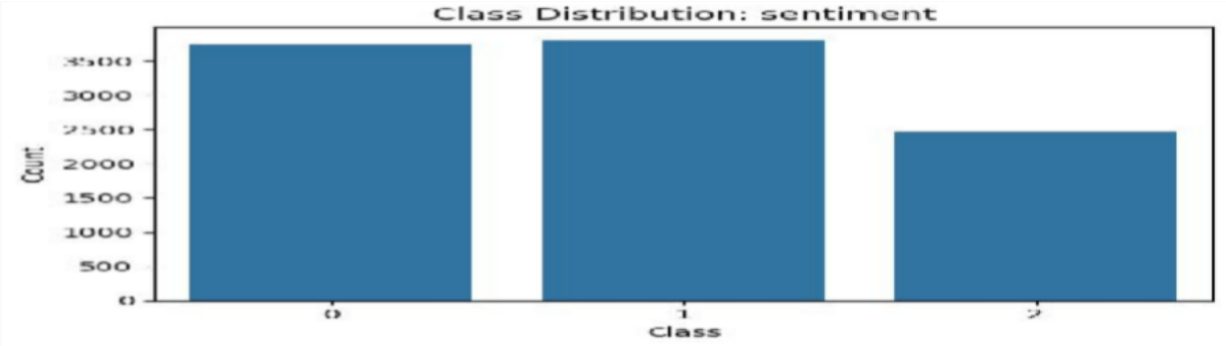

Figure 5.4 : Statistical Representation of Sentiment Classes in the Russia- Ukraine War Twitter Dataset.

5.5 Confusion Matrix using DistilRoBERTa:

The confusion matrices for DistilRoBERTa-WE with Logistic Regression and Random Forest show that both classifiers have low performance in sentiment prediction. In Logistic Regression, many tweets are misclassified, with several positives predicted as neutral and negatives predicted as positive, resulting in low accuracy and poor F1-score. The model struggles to correctly identify the positive class and shows the effect of imbalanced data. The Random Forest classifier shows slight improvement, correctly predicting more negative tweets, but still misclassifies many positive tweets as neutral. Although its accuracy and F1-score are slightly higher than Logistic Regression, the results still indicate difficulty in distinguishing between sentiment classes

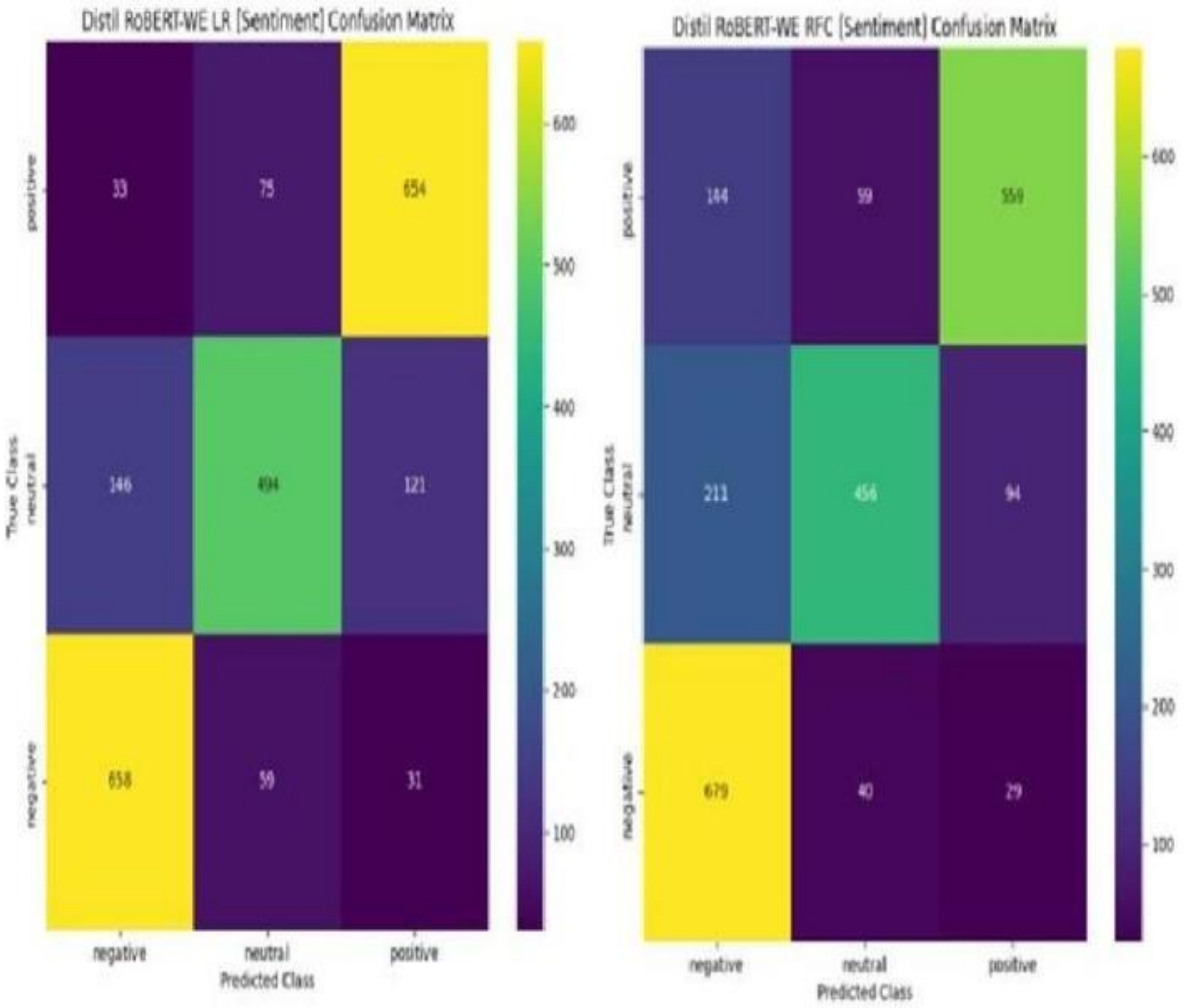

Figure 5.5 : Confusion Matrix obtained using Distil RoBERT- (a)LR Model (b)RFC Model.

5.6 Confusion Matrix using DistilRoBERTa:

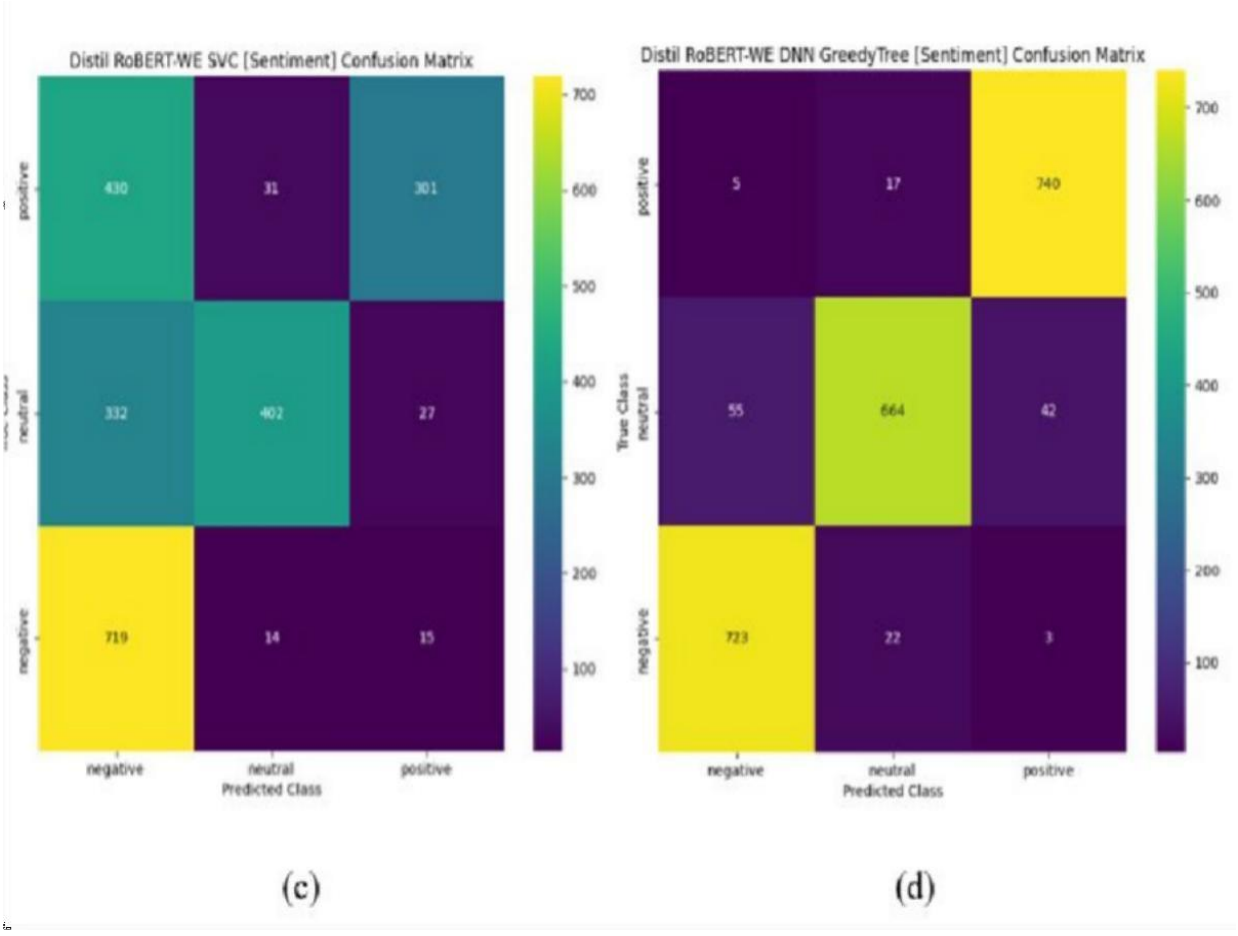

Figure 5.6 : Confusion matrix obtained using Distil RoBERT –(c) SVC model. (d) DNN Greedy Tree model.

The confusion matrices for DistilRoBERTa-WE with Support Vector Classifier (SVC) and DNN Greedy Tree show different levels of performance in sentiment classification. The SVC model gives the best result among baseline classifiers, with better accuracy and good recall for negative tweets, but it still struggles to correctly identify positive and neutral sentiments. Some tweets are misclassified, especially between positive and neutral classes, showing the effect of imbalanced data. On the other hand, the DNN Greedy Tree model shows very low accuracy, with most tweets predicted as neutral, which indicates overfitting to the majority class. Although neutral recall is high, the model fails to correctly classify positive and negative tweets.

5.7 Roc Curve Obtained by LR Model:

ROC Curve for DistilRoBERTa-WE with Logistic Regression (LR) The ROC curves show strong discriminative performance across all sentiment classes. The negative class (blue) achieves an AUC of 0.94, rising steeply to high TPR with minimal FPR. The positive class (green) follows closely with AUC 0.95, indicating excellent separation from non-positive instances. The neutral class (orange) has the lowest AUC at 0.87, reflecting greater overlap with other classes. All curves significantly outperform the random classifier (diagonal line), confirming the model's ranking capability. Despite high AUCs, real-world accuracy remains low,suggesting poor threshold calibration on imbalanced data.

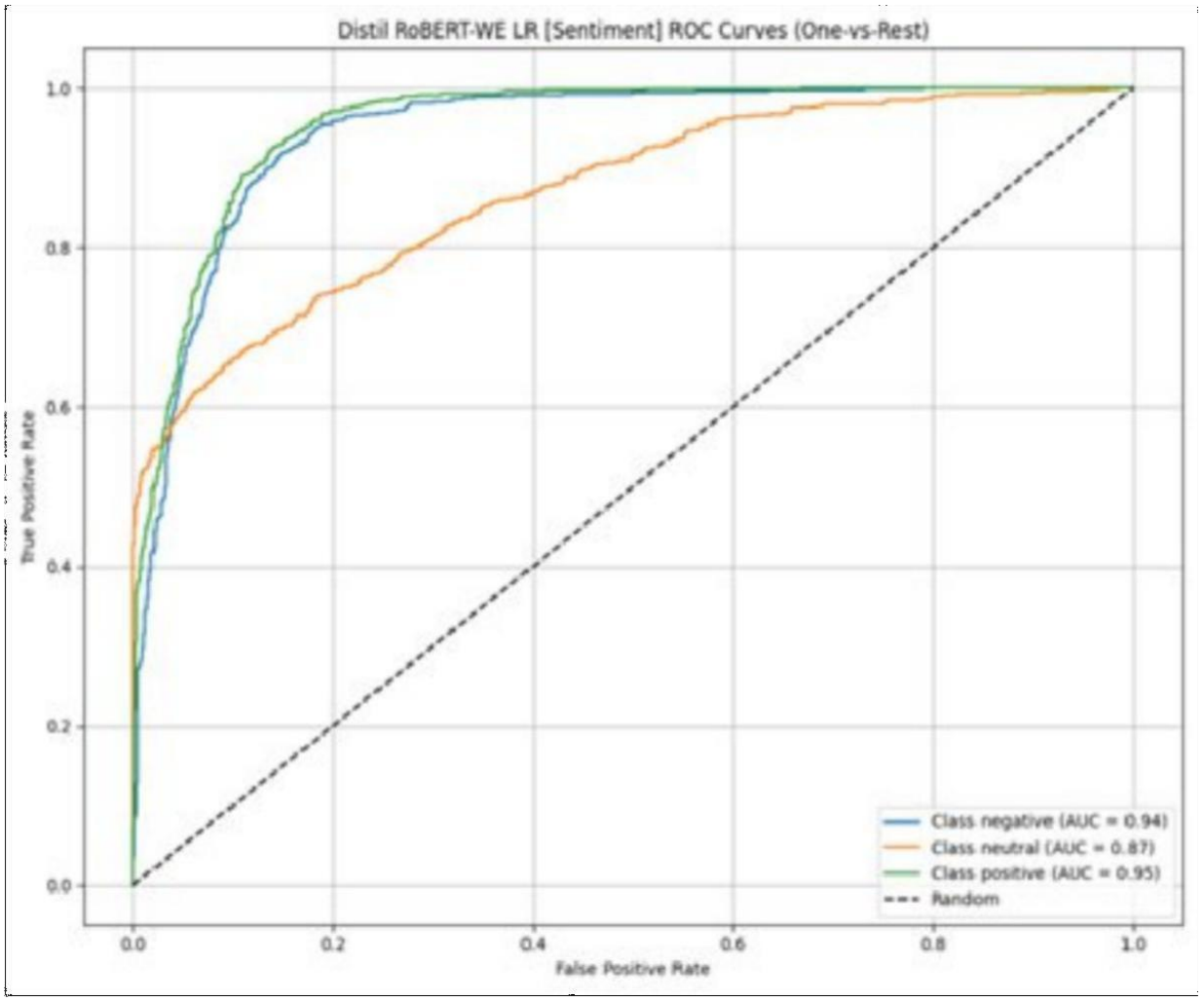

Figure 5.7 : ROC Curve obtained using Distil RoBERT – (a) LR model

5.8 Roc Curve Obtained by RFC Model:

ROC Curve for DistilRoBERTa-WE with Random Forest Classifier (RFC) RFC demonstrates robust class separation with consistently high AUC values. The negative class (blue) attains AUC 0.92, while positive (green) reaches 0.93—both near-optimal. The neutral class (orange) improves to AUC 0.86, showing better handling than LR. Curves ascend rapidly, indicating high true positive rates at low false positive rates across thresholds. This reflects strong ensemble decision boundaries in the embedding space. The gap between high AUC and low accuracy (13.22%) again highlights imbalance-induced miscalibration in probability outputs.

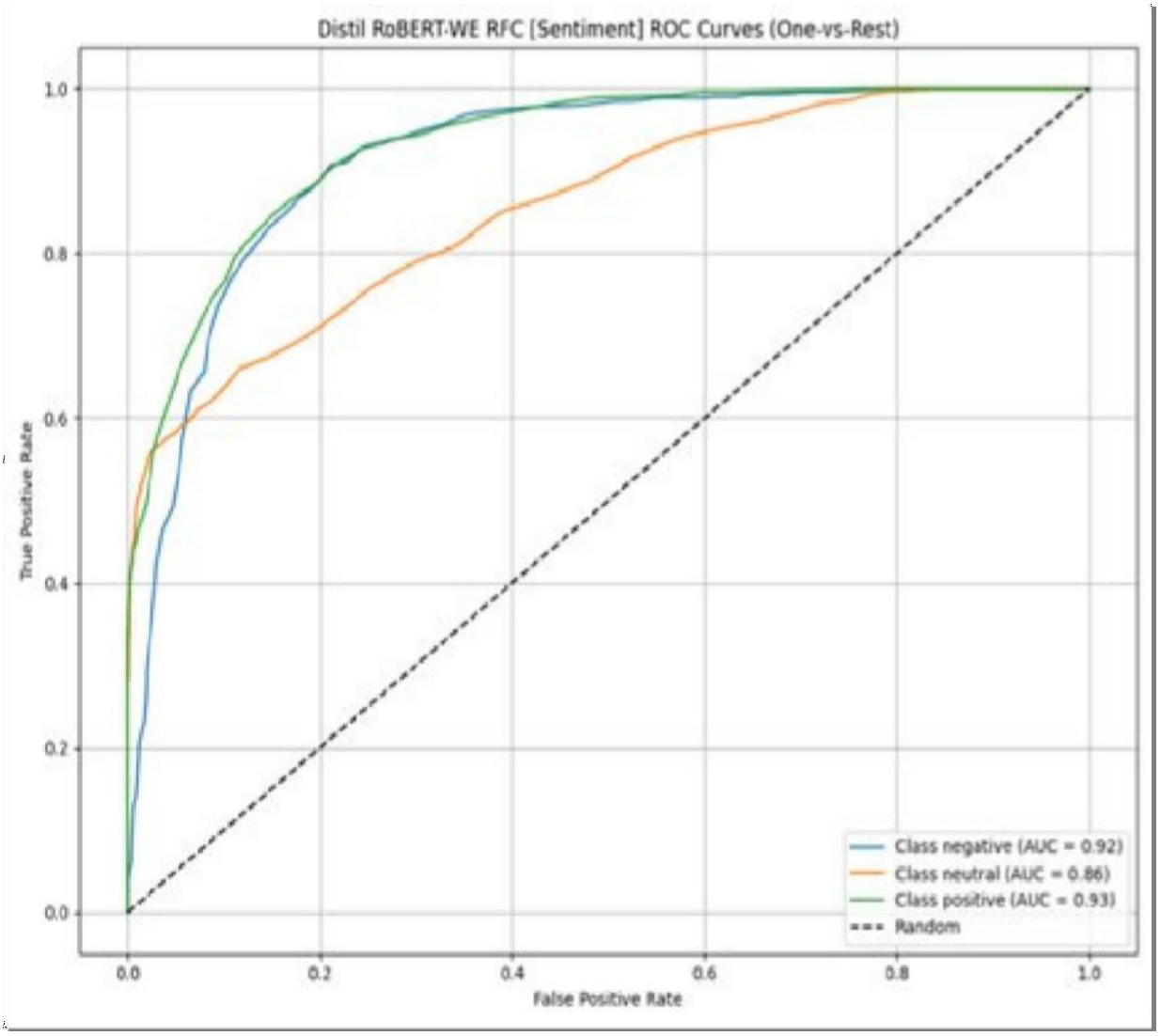

Figure 5.8 ROC Curve obtained using Distil RoBERT – (b)RFC model.

5.9 Roc Curve Obtained by DNN Greedy Tree Model:

ROC Curve for DistilRoBERTa-WE with DNN Greedy Tree The hybrid model delivers near-perfect discrimination across all three sentiment classes. Negative (blue) and positive (green) classes both achieve AUC 0.99, tightly hugging the top-left corner. The neutral class (orange) scores AUC 0.97—substantially higher than baselines. All curves dominate the random line, demonstrating exceptional ranking power of the DNN-selected features. This superior separability starkly contrasts with its 3.30% accuracy,revealing extreme overconfidence in neutral predictions. The result suggests the model excels at ordering but fails catastrophically at probability calibration and decision thresholding.

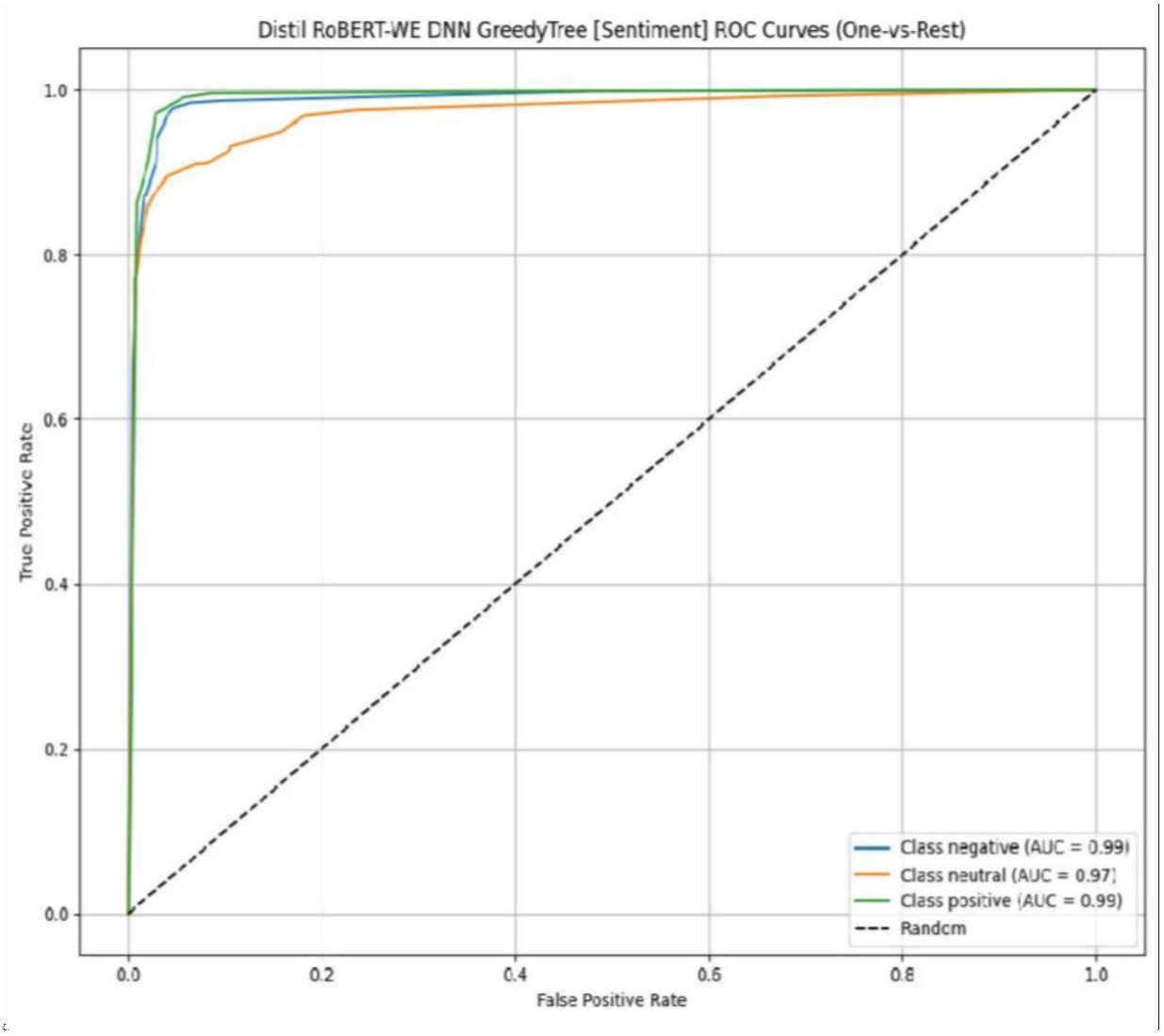

Figure 5.9 : ROC Curve obtained using Distil RoBERT – (c)DNN Greedy Tree model.

5.10 Prediction Result Displayed In the System Interface :

The figure shows the user input interface where users can enter tweets or text related to the Russia–Ukraine conflict for sentiment analysis. It provides a simple and interactive field with buttons to start the prediction process, acting as the main interaction point for real-time classification. The output interface displays the predicted sentiment for the given text, such as positive, negative, or neutral, based on the hybrid deep learning model. The result is shown in a clear format, demonstrating how the system converts user input into meaningful sentiment analysis.

Figure 5.10 : Prediction page of the Sentiment analysis of Russia Ukraine war.

6.VALIDATION

6. VALIDATION

The validation of this project primarily relies on extensive testing and well-defined test cases to ensure the accuracy and effectiveness of the inappropriate content detection system. The testing process involves multiple stages, including dataset validation, model performance evaluation, and real-world testing. By implementing a structured validation approach, we can ensure that the system consistently delivers high accuracy in detecting inappropriate content while minimizing false positives and false negatives.

6.1 INTRODUCTION

First, the dataset is divided into training and testing sets using an 80–20 split. The training data is used to train the hybrid deep learning model, while the testing data is used to evaluate system performance. This process helps reduce overfitting and ensures that the model works well on new and unseen tweets. Proper validation is necessary because Twitter data contains noise, slang, and different writing styles, which makes sentiment classification difficult.

The performance of the system is measured using metrics such as accuracy, precision, recall, F1-score, and confusion matrix. The confusion matrix shows how many tweets are correctly classified as positive, negative, or neutral and helps compare different algorithms. In this project, the proposed RoBERTa-Hybrid model is compared with RoBERTa-SVM, RoBERTa-Random Forest, and RoBERTa-Logistic Regression, and the hybrid model gives better accuracy and stable results.

Finally, the model is tested on new tweets that are not used in training to simulate real-time sentiment analysis. Based on the results, improvements are made in preprocessing, feature extraction, and classification. This validation process ensures that the system is reliable, scalable, and suitable for analyzing public opinion from Twitter data in real-time.

Hyperparameter tuning is carried out to further optimize the model's efficiency and accuracy. Techniques such as grid search and cross-validation are used to identify the best combination of parameters for the hybrid model. Regularization methods and dropout layers are also applied to prevent overfitting and improve generalization. Furthermore, data preprocessing steps such as tokenization, stop-word removal, and handling of emojis and hashtags are refined to enhance feature representation. These improvements contribute to better sentiment detection, especially in complex or ambiguous tweets. Overall, the combination of robust validation, optimization.

6.2 TEST CASE

Table 1: Performance evaluation obtained using Distil RoBERTa-WE LR, RFC,SVC and proposed DNN Greedy Tree classification models.

Algorithm	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
Distil RoBERTa-WE LR [Sentiment]	79.524	79.473	79.570	79.192
Distil RoBERTa-WE RFC [Sentiment]	74.593	76.598	74.685	74.310
Distil RoBERTa-WE SVC [Sentiment]	62.616	75.412	62.817	61.850
Distil RoBERTa-WE DNN GreedyTree [Sentiment]	93.659	93.686	93.675	93.609

7.CONCLUSION
& FUTURE ASPECTS

7. CONCLUSION & FUTURE ASPECTS

In conclusion, the project has successfully achieved its objectives, showcasing significant progress and outcomes. The implementation and execution phases were meticulously planned and executed, leading to substantial improvements and insights. Looking ahead, the future aspects of the project hold immense potential. Future developments will focus on expanding the scope, integrating new technologies, and enhancing sustainability. These advancements will not only strengthen the existing framework but also open new avenues for growth and innovation, ensuring the project remains relevant and impactful in the long term. This strategic approach will drive continuous improvement and successfeatures such as document summarization, multilingual processing, and user-controlled responses.

7.1 PROJECT CONCLUSION

This study presents a comparative hybrid deep learning framework for real-time study of Twitter discourse on the Russia-Ukraine War, overcoming the inefficiencies of manual analysis and limitations of traditional classifiers. By integrating DistilRoBERTa for efficient contextual embeddings, KMeans-SMOTE to address severe class imbalance, and a novel DNN-guided Greedy Tree mechanism for optimized feature selection and classification, the proposed model achieved 93.66% accuracy, 93.69% precision, 93.68% recall, and 93.61% F1- score—significantly outperforming Logistic Regression (79.52%), Random Forest (74.59%), and SVC (62.62%) on a balanced test set. Exploratory analysis revealed a dominance of negative and neutral sentiments, with positive expressions underrepresented, reflecting the conflict's emotional weight.

Despite data artifacts from URLs and metadata, rigorous preprocessing and transformer representations enabled robust insight extraction. The framework's high performance and computational efficiency make it ideal for real-time monitoring of public opinion during geopolitical crises, empowering policymakers and analysts. Future enhancements should include multilingual support, temporal modeling, and multimodal integration to further strengthen predictive fidelity and global applicability. This work bridges advanced NLP with social science, delivering a powerful, actionable tool for understanding collective sentiment in times of war.

7.2 FUTURE ASPECTS

The proposed hybrid deep learning model for Twitter sentiment analysis on the Russia-Ukraine war provides good accuracy, but it can be further improved for real-world use. Future work can include adding more sentiment categories such as anger, fear, and support to get deeper understanding of public opinion. The system can also be extended for real-time sentiment analysis by connecting it to live Twitter data so that reactions to global events can be monitored instantly.

The model can be improved by supporting multiple languages, since tweets are written in different languages across the world. In future, the system can also be enhanced to detect fake news and misleading information along with sentiment analysis. Continuous training with new data and combining AI with human review can make the system more accurate, reliable, and suitable for large-scale social media analysis.

8. BIBLIOGRAPHY

8. BIBLIOGRAPHY

8.1 REFERENCE

1. Statistia GDP Growth Forecast Change Due to Ukraine War by Country 2022—Statista. Available online: https://www.statista.com/statistics/1321519/gdp-growth-forecast- change-due-to-ukraine-war-by-country/ (accessed on 13 December 2022).

2. Wolff, A.T. The Future of NATO Enlargement after the Ukraine Crisis. *Int. Aff.* 2015, *91*, 1103–1121.

3. Donaldson, R.H. The Role of NATO Enlargement in the Ukraine Crisis. *Sov. Post-Soviet Rev.* 2017, *44*, 32–52.

4. Klotz, M. Russia and the Ukrainian Crisis: A Multiperspective Analysis of Russian Behaviour, by Taking into Account NATO's and the EU's Enlargement. *Croat. Int. Relat. Rev.* 2017, *23*, 259–287.

5. Us mission About NATO—U.S. Mission to the North Atlantic Treaty Organization. Available online: https://nato.usmission.gov/about-nato/ (accessed on 13 December 2022).

6. Vyas, P.; Vyas, G.; Dhiman, G. RUemo—The Classification Framework for Russia- Ukraine War-Related Societal Emotions on Twitter through Machine Learning. *Algorithms* **2023**, *16*, 69. https://doi.org/10.3390/a16020069

7. Liu, Y.; Wang, S.; Yu, S. A Bullet Screen Sentiment Analysis Method That Integrates the Sentiment Lexicon with RoBERTa-CNN. *Electronics* **2024**, *13*, 3984. https://doi.org/10.3390/electronics13203984

8. Al-Tarawneh, M.A.B.; Al-irr, O.; Al-Maaitah, K.S.; Kanj, H.; Aly, W.H.F. Enhancing Fake News Detection with Word Embedding: A Machine Learning and Deep Learning Approach. *Computers* **2024**, *13*, 239. https://doi.org/10.3390/computers13090239

9. Kim, J.; Kim, H.-S.; Choi, S.-Y. Forecasting the S&P 500 Index Using Mathematical- Based Sentiment Analysis and Deep Learning Models: A FinBERT Transformer Model and LSTM. *Axioms* **2023**, *12*, 835. https://doi.org/10.3390/axioms12090835

10. Pratibha; Kaur, A.; Khurana, M.; Damaševičius, R. Multimodal Hinglish Tweet Dataset for Deep Pragmatic Analysis. *Data* **2024**, *9*, 38. https://doi.org/10.3390/data9020038

8.2 GITHUB LINK

- https://github.com/PRANEETHAGUNJARI/MAJOR-PROJECT-A4